DEEP LEARNING LABORATORY

Python-Based Scenario-Oriented Experiments with Datasets

Course Outcome: CO2 | Total Experiments: 23

Instructions: For each experiment, students should write and execute a Python program, show relevant outputs and visualizations, and briefly interpret the results. Use NumPy for foundational mathematical implementations and TensorFlow/Keras for neural-network and CNN experiments.

Phase 1: Neural Network Fundamentals

Experiment 1: Single-Layer Perceptron for Binary Classification
Dataset: AND Logic Gate dataset
Python Programming Question: Write a Python program using NumPy to implement a Single-Layer Perceptron for binary classification using the AND Logic Gate dataset. Initialize weights and bias, implement the step activation function, train the perceptron using the weight-update rule, and display actual and predicted outputs.

Concept: Perceptron | CO2

AIM:

To implement a Single-Layer Perceptron using NumPy to classify the AND Logic Gate dataset.

DATASET:

AND Logic Gate Dataset

Input 1 (X1)	Input 2 (X2)	Actual Output (Y)
0	**0**	**0**
0	**1**	**0**
1	**0**	**0**
1	**1**	**1**

PROCEDURE:

- Import the NumPy library.
- Create the AND Logic Gate dataset.
- Initialize weights and bias with zeros.
- Define the step activation function.
- Train the perceptron using the weight-update rule.
- Display actual and predicted outputs.

PROGRAM CODE:

```
import numpy as np
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([0, 0, 0, 1])
weights = np.zeros(2)
bias = 0
learning_rate = 0.1
epochs = 10
def step(x):
    return 1 if x >= 0 else 0
for epoch in range(epochs):
    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        prediction = step(net)
        error = y[i] - prediction
        weights += learning_rate * error * X[i]
        bias += learning_rate * error
print("Actual Output | Predicted Output")
for i in range(len(X)):
    net = np.dot(X[i], weights) + bias
    prediction = step(net)
    print("     ", y[i], "      |       ", prediction)
print("\nFinal Weights:", weights)
print("Final Bias:", bias)
```

INPUT:

- (0, 0) → 0
- (0, 1) → 0
- (1, 0) → 0
- (1, 1) → 1
- Learning rate = 0.1
- Epochs = 10

OUTPUT:

```
print("Final Bias:", bias)


Actual Output | Predicted Output
       0      |        0
       0      |        0
       0      |        0
       1      |        1

Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004
```

RESULT:

The Single-Layer Perceptron successfully learned the AND Logic Gate and produced the correct binary outputs.

Experiment 2: Feedforward Neural Network for the XOR Problem
Dataset: XOR Logic Gate dataset
Python Programming Question: Write a Python program using NumPy or TensorFlow/Keras to implement a feedforward neural network for solving the XOR problem. Design input, hidden, and output layers, train the network, and display predictions for all four input combinations.

Concept: Feedforward Neural Networks | CO2

AIM:

To implement a Feedforward Neural Network using NumPy to solve the XOR Logic Gate problem.

DATASET:

INPUT: XOR Dataset

Input 1	Input 2	Actual Output
0	0	0
0	1	1
1	0	1
1	1	0

PROCEDURE:

- Import the NumPy library.
- Create the XOR Logic Gate dataset.
- Initialize weights and biases.
- Design input, hidden, and output layers.
- Apply the sigmoid activation function.
- Train the network using backpropagation.
- Display actual and predicted outputs.

PROGRAM CODE:

```
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
```

```
y = np.array([[0],
              [1],
              [1],
              [0]])
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
np.random.seed(42)
W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
learning_rate = 0.5
epochs = 20000
for epoch in range(epochs):
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)
    error = y - predicted_output
    d_output = error * sigmoid_derivative(predicted_output)
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(hidden_output)
    W2 += np.dot(hidden_output.T, d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += np.dot(X.T, d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate
print("Input | Actual | Predicted")
for i in range(len(X)):
```

h = sigmoid(np.dot(X[i], W1) + b1)

output = sigmoid(np.dot(h, W2) + b2)

prediction = 1 if output[0][0] >= 0.5 else 0

print(X[i], " | ", y[i][0], " | ", prediction)

INPUT & OUTPUT:

```
        output = sigmoid(np.dot(h, W2) + b2)
        prediction = 1 if output[0][0] >= 0.5 else 0

        print(X[i], " | ", y[i][0], " | ", prediction)
```

```
Input | Actual | Predicted
[0 0]  |  0  |  0
[0 1]  |  1  |  1
[1 0]  |  1  |  1
[1 1]  |  0  |  0
```

RESULT:

The Feedforward Neural Network successfully learned the XOR logic gate and predicted all four outputs correctly.

Experiment 3: Multilayer Neural Network for Iris Classification
Dataset: Iris Dataset (UCI Machine Learning Repository)

Python Programming Question: Write a Python program using TensorFlow/Keras to build a multilayer neural network for classifying flower species using the Iris Dataset. Perform preprocessing, training, prediction, and evaluation using accuracy and a confusion matrix.

Concept: Multilayer Networks | CO2

AIM:

To implement a Multilayer Neural Network using TensorFlow/Keras to classify Iris flower species and evaluate its performance using accuracy and a confusion matrix.

DATASET:

Iris Dataset

The Iris dataset contains 150 flower samples with 4 input features and 3 output classes.

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	Setosa
4.9	3.0	1.4	0.2	Setosa
6.3	3.3	6.0	2.5	Virginica
5.8	2.7	5.1	1.9	Virginica
5.5	2.4	3.8	1.1	Versicolor

PROCEDURE:

- Import the required libraries.
- Load the Iris dataset.
- Separate input features and target labels.
- Preprocess the data using StandardScaler.
- Split the dataset into training and testing sets.
- Build a multilayer neural network using Keras.
- Train the model using the training dataset.
- Predict flower species using the test dataset.
- Evaluate accuracy and display the confusion matrix.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_reportiris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(4,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(X_train, y_train, epochs=100, batch_size=8, verbose=0)
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
```

```
y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)
print("Actual Labels:   ", y_test)
print("Predicted Labels:", y_pred)
print("\nAccuracy:", accuracy_score(y_test, y_pred) * 100, "%")
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                            target_names=iris.target_names))
plt.imshow(cm, cmap='Blues')
plt.title("Iris Classification Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("Actual Labels")
plt.xticks([0, 1, 2], iris.target_names)
plt.yticks([0, 1, 2], iris.target_names)
plt.colorbar()
plt.show()
```

INPUT & OUTPUT:

```
Actual Labels:    [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]
Predicted Labels: [0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

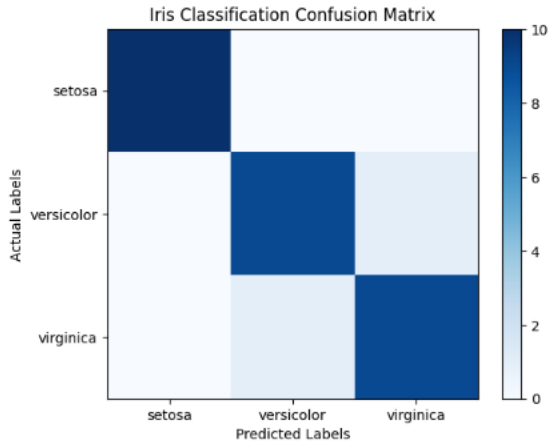
Accuracy: 93.33333333333333 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.90      0.90      0.90        10
   virginica       0.90      0.90      0.90        10

    accuracy                           0.93        30
   macro avg       0.93      0.93      0.93        30
weighted avg       0.93      0.93      0.93        30
```



RESULT:

The Multilayer Neural Network successfully classifies Iris flower species and evaluates performance using accuracy and a confusion matrix.

Experiment 4: Implementation of Sigmoid, Tanh, and ReLU Activation Functions
Dataset: Numerical input values (no external dataset required)

Python Programming Question: Write a Python program using NumPy and Matplotlib to implement Sigmoid, Tanh, and ReLU activation functions. Generate input values, calculate the corresponding activation outputs, and plot their graphs for comparison.

Concept: Activation Functions | CO2

AIM:

To implement Sigmoid, Tanh, and ReLU activation functions using NumPy and Matplotlib and compare their outputs through graphs.

DATASET:

(Numerical Input Values)

No external dataset is required. Use the following numerical input values.

S.No	Input (X)
1	-10
2	-8
3	-6
4	-4
5	-2
6	0
7	2
8	4
9	6
10	8
11	10

PROCEDURE:

- Import NumPy and Matplotlib libraries.
- Generate numerical input values from -10 to 10.
- Implement the Sigmoid activation function.
- Implement the Tanh activation function.
- Implement the ReLU activation function.
- Calculate activation outputs.
- Plot all three activation functions for comparison.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)

sigmoid = 1 / (1 + np.exp(-x))

tanh = np.tanh(x)

relu = np.maximum(0, x)

print("Input\tSigmoid\t\tTanh\t\tReLU")

for i in range(0, 100, 10):

    print(f"{x[i]:.2f}\t{sigmoid[i]:.4f}\t\t{tanh[i]:.4f}\t\t{relu[i]:.2f}")

plt.figure(figsize=(10, 6))

plt.plot(x, sigmoid, label="Sigmoid")

plt.plot(x, tanh, label="Tanh")

plt.plot(x, relu, label="ReLU")

plt.title("Comparison of Activation Functions")

plt.xlabel("Input Values")

plt.ylabel("Activation Output")

plt.legend()

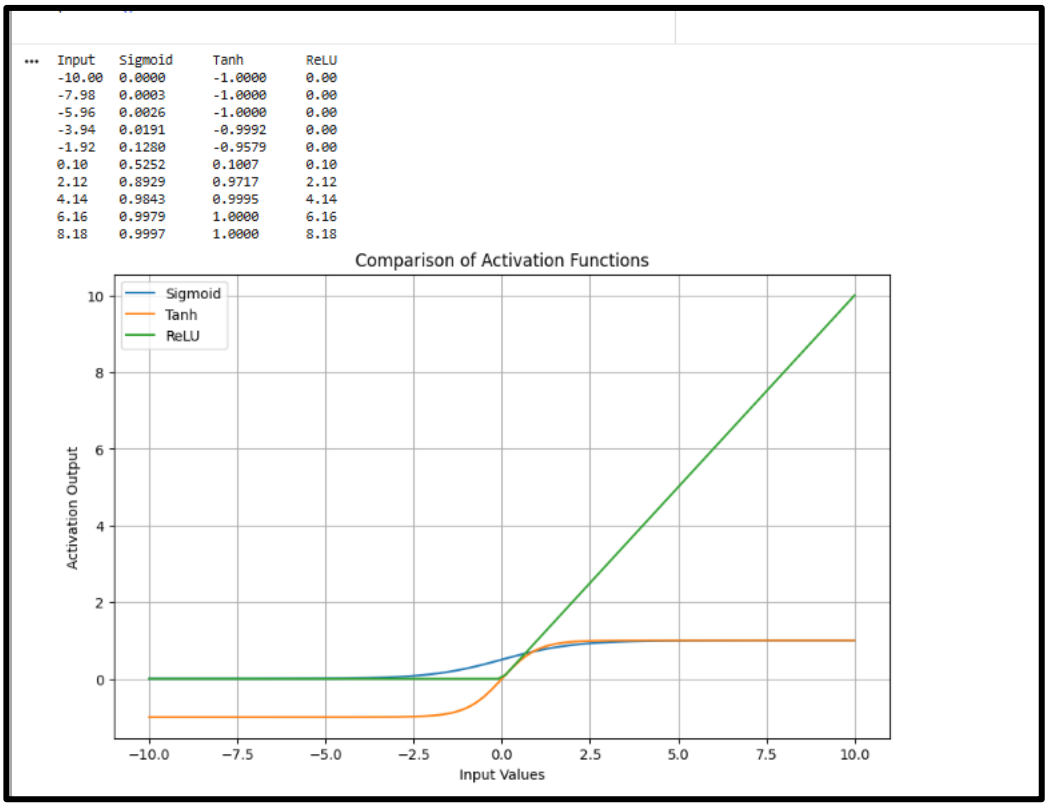
plt.grid(True)

plt.show()
```

INPUT:

- Numerical input values: -10 to 10
- Number of values: 100
- Activation functions: Sigmoid, Tanh, ReLU

OUTPUT:



RESULT:

Sigmoid, Tanh, and ReLU activation functions were successfully implemented and plotted using NumPy and Matplotlib.

Experiment 5: Comparison of ReLU, Sigmoid, and Tanh Activations
Dataset: Breast Cancer Wisconsin Diagnostic Dataset (Scikit-learn)

Python Programming Question: Write a Python program using TensorFlow/Keras to compare ReLU, Sigmoid, and Tanh activation functions. Train three neural networks with identical architectures and compare training accuracy, validation accuracy, and loss.

Concept: Activation Function Analysis | CO2

AIM:

To compare ReLU, Sigmoid, and Tanh activation functions using TensorFlow/Keras on the Breast Cancer Wisconsin Diagnostic Dataset by evaluating training accuracy, validation accuracy, and loss.

DATASET:

The dataset contains 569 samples, 30 input features, and 2 output classes.

Dataset Classes:

1. Malignant (0)
2. Benign (1)

Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Diagnosis
17.99	10.38	122.80	1001.0	Malignant
20.57	17.77	132.90	1326.0	Malignant
19.69	21.25	130.00	1203.0	Malignant
11.42	20.38	77.58	386.1	Benign
20.29	14.34	135.10	1297.0	Malignant

PROCEDURE:

- Import the required libraries.
- Load the Breast Cancer Wisconsin dataset.
- Split the dataset into training and testing sets.
- Normalize the input features using StandardScaler.
- Build three neural networks with identical architectures.
- Use ReLU, Sigmoid, and Tanh as hidden-layer activation functions.
- Train all three models using the same settings.
- Compare training accuracy, validation accuracy, and loss.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
activations = ['relu', 'sigmoid', 'tanh']
results = {}
for activation in activations:
    tf.keras.utils.set_random_seed(42)
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(30,)),
        tf.keras.layers.Dense(16, activation=activation),
        tf.keras.layers.Dense(8, activation=activation),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
```

```
        metrics=['accuracy']
    )
    history = model.fit(
        X_train, y_train,
        epochs=50,
        batch_size=16,
        validation_split=0.2,
        verbose=0
    )
    results[activation] = history
    print("\nActivation:", activation)
    print("Training Accuracy:",
          history.history['accuracy'][-1] * 100, "%")
    print("Validation Accuracy:",
          history.history['val_accuracy'][-1] * 100, "%")
    print("Validation Loss:",
          history.history['val_loss'][-1])
for activation in activations:
    plt.plot(results[activation].history['val_accuracy'],
             label=activation)
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
for activation in activations:
    plt.plot(results[activation].history['val_loss'],
             label=activation)
```

plt.title("Validation Loss Comparison")

plt.xlabel("Epochs")

plt.ylabel("Loss")

plt.legend()

plt.grid()

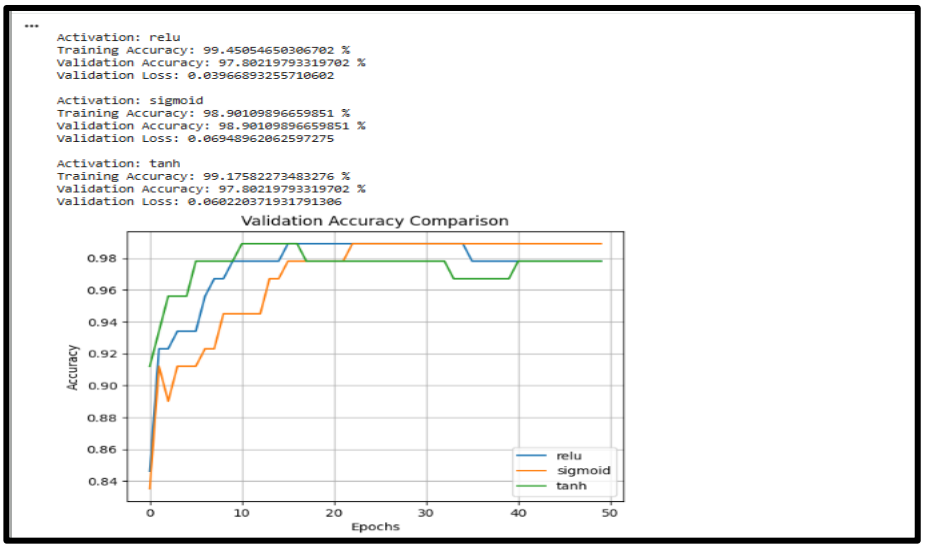
plt.show()

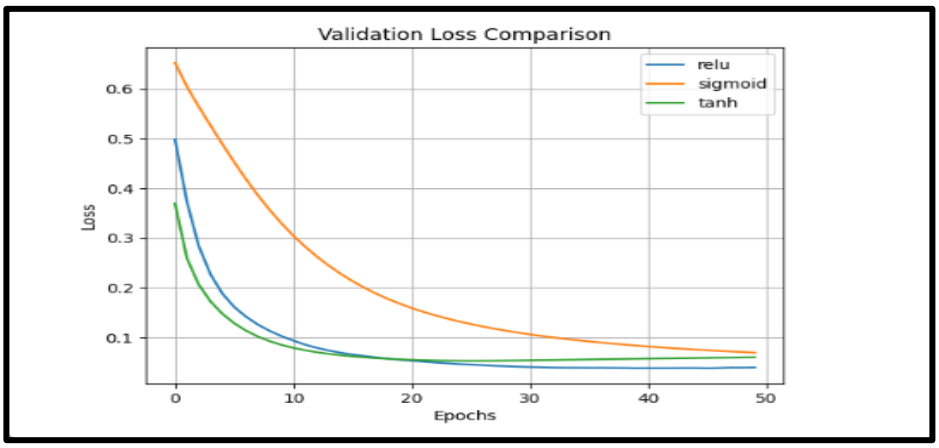
INPUT & OUTPUT:

```
...
    Activation: relu
    Training Accuracy: 99.45054650306702 %
    Validation Accuracy: 97.80219793319702 %
    Validation Loss: 0.03966893255710602

    Activation: sigmoid
    Training Accuracy: 98.90109896659851 %
    Validation Accuracy: 98.90109896659851 %
    Validation Loss: 0.06948962062597275

    Activation: tanh
    Training Accuracy: 99.17582273483276 %
    Validation Accuracy: 97.80219793319702 %
    Validation Loss: 0.060220371931791306
```





RESULT:

ReLU, Sigmoid, and Tanh activation functions were successfully compared using neural networks, and their training accuracy, validation accuracy, and loss were evaluated.

Experiment 6: Forward Propagation in a Multilayer Neural Network
Dataset: Small manually generated numerical dataset (NumPy)

Python Programming Question: Write a Python program using NumPy to implement forward propagation in a multilayer neural network. Initialize weights and biases, calculate weighted sums, apply activation functions at each layer, and display intermediate and final outputs.

Concept: Forward Propagation | CO2

AIM:

To implement forward propagation in a multilayer neural network using NumPy by calculating weighted sums, applying activation functions, and displaying intermediate and final outputs.

DATASET:

Dataset

Sample	Input 1	Input 2
1	0.5	0.2
2	0.3	0.8
3	0.9	0.4

PROCEDURE:

- Import the NumPy library.
- Create a small numerical dataset.
- Initialize weights and biases for each layer.
- Calculate the weighted sum of the input layer.
- Apply the ReLU activation function to the hidden layer.
- Calculate the output layer values.
- Apply the Sigmoid activation function.
- Display intermediate and final outputs.

PROGRAM CODE:

```
import numpy as np

X = np.array([[0.5, 0.2],
              [0.3, 0.8],
              [0.9, 0.4]])

W1 = np.array([[0.2, 0.4],
               [0.3, 0.5]])
```

```
b1 = np.array([0.1, 0.1])
W2 = np.array([[0.6],
               [0.7]])
b2 = np.array([0.2])
def relu(x):
    return np.maximum(0, x)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
z1 = np.dot(X, W1) + b1
a1 = relu(z1)
z2 = np.dot(a1, W2) + b2
a2 = sigmoid(z2)
print("Input Dataset:")
print(X)
print("\nHidden Layer Weighted Sum:")
print(z1)
print("\nHidden Layer Output (ReLU):")
print(a1)
print("\nOutput Layer Weighted Sum:")
print(z2)
print("\nFinal Output (Sigmoid):")
print(a2)
```

INPUT & OUTPUT:

```
Input Dataset:
[[0.5 0.2]
 [0.3 0.8]
 [0.9 0.4]]

Hidden Layer Weighted Sum:
[[0.26 0.4 ]
 [0.4  0.62]
 [0.4  0.66]]

Hidden Layer Output (ReLU):
[[0.26 0.4 ]
 [0.4  0.62]
 [0.4  0.66]]

Output Layer Weighted Sum:
[[0.636]
 [0.874]
 [0.902]]

Final Output (Sigmoid):
[[0.6538487 ]
 [0.70557733]
 [0.71136033]]
```

RESULT:

Forward propagation in a multilayer neural network was successfully implemented using NumPy, and intermediate and final outputs were calculated.

Experiment 7: Backpropagation to Update Network Weights
Dataset: XOR Logic Gate dataset
Python Programming Question: Write a Python program using NumPy to implement backpropagation in a multilayer neural network. Calculate prediction errors, compute gradients using the chain rule, update weights and biases, and display loss reduction during training.

Concept: Backpropagation | CO2

AIM:

To implement backpropagation in a multilayer neural network using NumPy and update weights and biases to reduce prediction errors during training.

DATASET:

XOR Dataset

Input 1	Input 2	Actual Output
0	0	0
0	1	1
1	0	1
1	1	0

PROCEDURE:

- Import the NumPy library.
- Create the XOR Logic Gate dataset.
- Initialize weights and biases.
- Apply the sigmoid activation function.
- Perform forward propagation.
- Calculate prediction errors and loss.
- Compute gradients using the chain rule.
- Update weights and biases using gradient descent.
- Display loss reduction during training.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])
y = np.array([[0],
              [1],
              [1],
              [0]])
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x)
np.random.seed(42)
W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
learning_rate = 0.5
epochs = 10000
loss_history = []
for epoch in range(epochs):
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)
    error = y - output
    loss = np.mean(error ** 2)
    loss_history.append(loss)
```

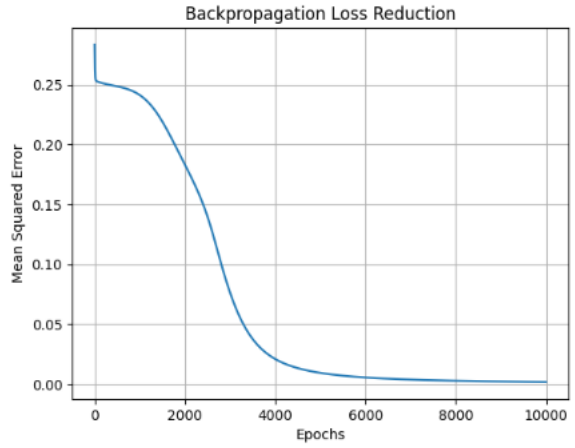
```
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(hidden)
    W2 += learning_rate * np.dot(hidden.T, d_output) / len(X)
    b2 += learning_rate * np.mean(d_output, axis=0, keepdims=True)
    W1 += learning_rate * np.dot(X.T, d_hidden) / len(X)
    b1 += learning_rate * np.mean(d_hidden, axis=0, keepdims=True)
print("Actual Output:")
print(y)
print("\nPredicted Output:")
print(np.round(output))
print("\nInitial Loss:", loss_history[0])
print("Final Loss:", loss_history[-1])
plt.plot(loss_history)
plt.title("Backpropagation Loss Reduction")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error")
plt.grid()
plt.show()
```

INPUT & OUTPUT:

```
Actual Output:
[[0]
 [1]
 [1]
 [0]]

Predicted Output:
[[0.]
 [1.]
 [1.]
 [0.]]

Initial Loss: 0.2831895890644397 5
Final Loss: 0.00176997059728368
```



RESULT:

Backpropagation was successfully implemented using NumPy. Network weights and biases were updated, reducing prediction errors during training.

Experiment 8: Neural Network Training Using Mean Squared Error Loss
Dataset: California Housing Dataset (Scikit-learn)

Python Programming Question: Write a Python program using TensorFlow/Keras to train a neural network for regression using the California Housing Dataset. Use Mean Squared Error as the loss function and compare actual and predicted house prices using graphs.

Concept: Loss Functions | CO2

AIM:

To train a neural network using TensorFlow/Keras for house price prediction with Mean Squared Error (MSE) loss and compare actual and predicted house prices using graphs.

DATASET:

Sample Dataset:

MedInc	HouseAge	AveRooms	AveBedrms	Population	House Price
8.325	41	6.98	1.02	322	4.526
8.301	21	6.24	0.97	2401	3.585
7.257	52	8.29	1.07	496	3.521
5.643	52	5.82	1.07	558	3.413
3.846	52	6.28	1.08	565	3.422

PROCEDURE:

- Import the required libraries.
- Load the California Housing dataset.
- Separate input features and target values.
- Split the dataset into training and testing sets.
- Normalize the input features using StandardScaler.
- Build a neural network using Keras.
- Compile the model using Adam optimizer and MSE loss.
- Train the model using training data.
- Predict house prices using testing data.
- Compare actual and predicted prices using a graph.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
data = fetch_california_housing()
X = data.data
y = data.target
print("Dataset Shape:", X.shape)
print("\nFirst 5 Input Samples:")
print(X[:5])
print("\nActual House Prices:")
print(y[:5])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(8,)),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(1)
])
model.compile(
```

```
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mae']
)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=0
)
y_pred = model.predict(X_test, verbose=0).flatten()
mse = mean_squared_error(y_test, y_pred)
print("\nActual Prices:", y_test[:10])
print("\nPredicted Prices:", np.round(y_pred[:10], 2))
print("\nMean Squared Error:", round(mse, 4))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("MSE Loss During Training")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.grid()
plt.show()
plt.figure(figsize=(8, 5))
plt.scatter(y_test[:100], y_pred[:100])
plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted House Prices")
```

plt.grid()

plt.show()

INPUT & OUTPUT:

```
...  Dataset Shape: (20640, 8)

     First 5 Input Samples:
     [[ 8.32520000e+00  4.10000000e+01  6.98412698e+00  1.02380952e+00
        3.22000000e+02  2.55555556e+00  3.78800000e+01 -1.22230000e+02]
      [ 8.30140000e+00  2.10000000e+01  6.23813708e+00  9.71880492e-01
        2.40100000e+03  2.10984183e+00  3.78600000e+01 -1.22220000e+02]
      [ 7.25740000e+00  5.20000000e+01  8.28813559e+00  1.07344633e+00
        4.96000000e+02  2.80225989e+00  3.78500000e+01 -1.22240000e+02]
      [ 5.64310000e+00  5.20000000e+01  5.81735160e+00  1.07305936e+00
        5.58000000e+02  2.54794521e+00  3.78500000e+01 -1.22250000e+02]
      [ 3.84620000e+00  5.20000000e+01  6.28185328e+00  1.08108108e+00
        5.65000000e+02  2.18146718e+00  3.78500000e+01 -1.22250000e+02]]

     Actual House Prices:
     [4.526 3.585 3.521 3.413 3.422]

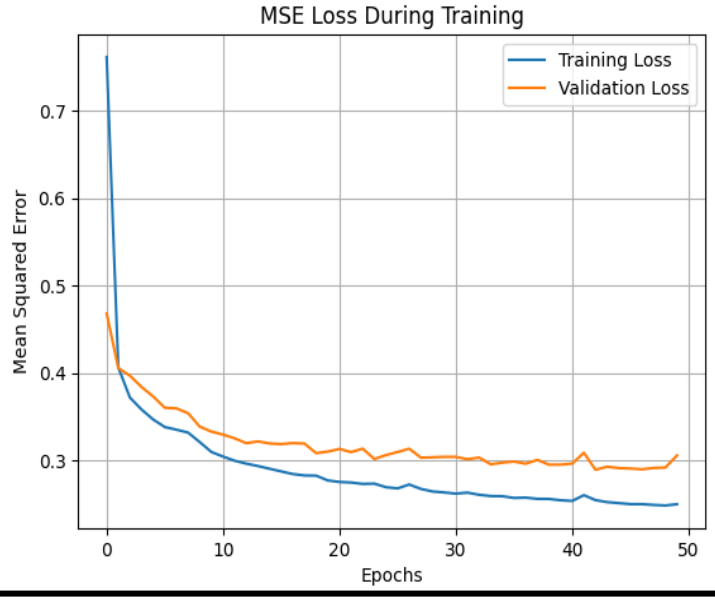
     Actual Prices: [0.477   0.458   5.00001 2.186   2.78    1.587   1.982   1.575   3.4
      4.466  ]

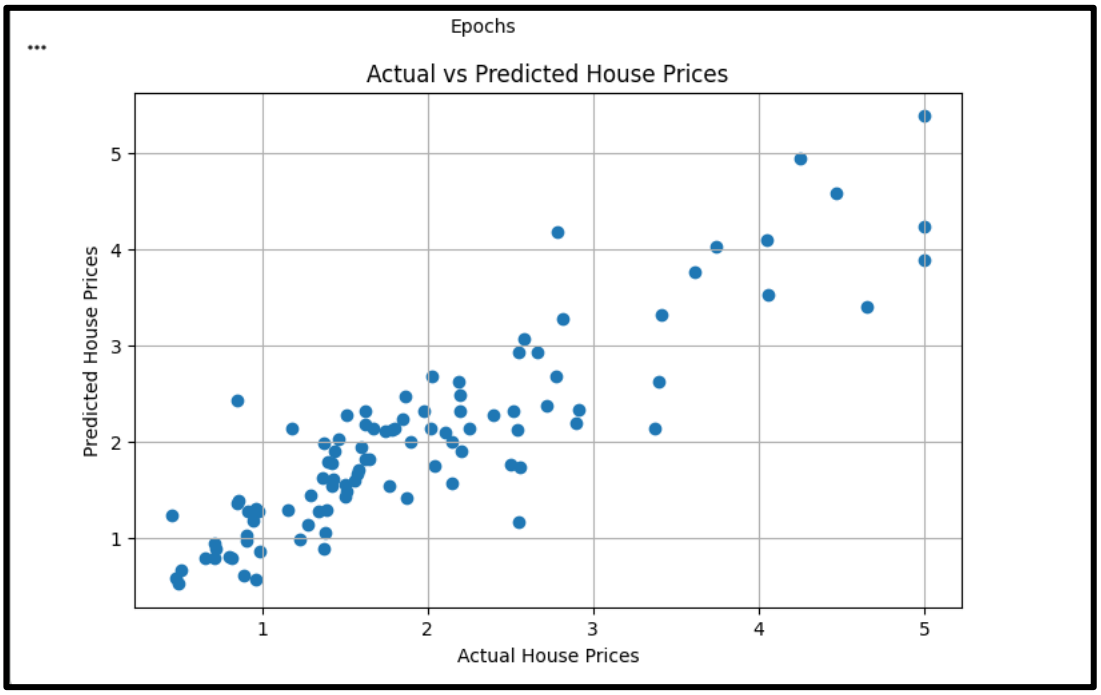
     Predicted Prices: [0.58 1.24 5.38 2.62 2.68 1.71 2.31 1.66 2.62 4.58]

     Mean Squared Error: 0.2806
```

```
Predicted Prices: [0.58 1.24 5.38 2.62 2.68 1.71 2.31 1.66 2.62 4.58]

Mean Squared Error: 0.2806
```





RESULT:

The neural network was successfully trained using MSE loss to predict California house prices and compare actual and predicted values graphically. The neural network was successfully trained using MSE loss to predict California house prices and compare actual and predicted values graphically.

Experiment 9: Neural Network Training Using Binary Cross-Entropy Loss
Dataset: Pima Indians Diabetes Dataset (UCI/Kaggle)

Python Programming Question: Write a Python program using TensorFlow/Keras to develop a binary classification neural network using the Pima Indians Diabetes Dataset. Use Binary Cross-Entropy loss and Sigmoid activation, and evaluate the model using accuracy, precision, recall, and F1-score.

Concept: Binary Classification | CO2

AIM:

To develop a binary classification neural network using TensorFlow/Keras on the Pima Indians Diabetes Dataset with Binary Cross-Entropy loss and evaluate its performance using accuracy, precision, recall, and F1-score.

DATASET:

Sample Dataset:

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Age	Outcome
6	148	72	35	0	33.6	50	1
1	85	66	29	0	26.6	31	0
8	183	64	0	0	23.3	32	1
1	89	66	23	94	28.1	21	0
0	137	40	35	168	43.1	33	1

PROCEDURE:

- Import the required libraries.
- Load the Pima Indians Diabetes dataset.
- Separate input features and target labels.
- Replace invalid zero values in selected medical measurements.
- Split the dataset into training and testing sets.
- Normalize the input features using StandardScaler.
- Build a neural network with hidden layers and Sigmoid output activation.
- Compile the model using Binary Cross-Entropy loss.
- Train the model using the training dataset.
- Predict diabetes outcomes and calculate accuracy, precision, recall, and F1-score.

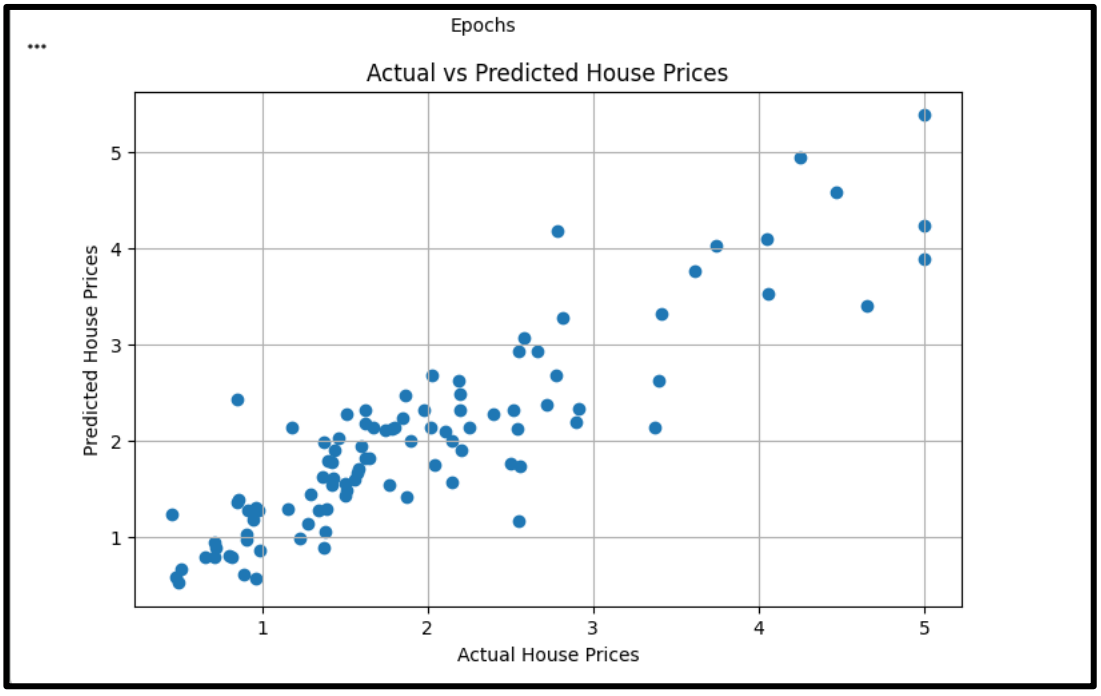
PROGRAM CODE:

```
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
uploaded = files.upload()
df = pd.read_csv(next(iter(uploaded)))
print("Dataset:")
print(df.head())
X = df.drop('Outcome', axis=1)
y = df['Outcome']columns = ['Glucose', 'BloodPressure',
        'SkinThickness', 'Insulin', 'BMI']
for col in columns:
    X[col] = X[col].replace(0, np.nan)
    X[col] = X[col].fillna(X[col].median())
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(8,)),
    tf.keras.layers.Dense(16, activation='relu'),
```

```
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)
y_prob = model.predict(X_test, verbose=0)
y_pred = (y_prob >= 0.5).astype(int).flatten()
print("\nModel Evaluation")
print("Accuracy:", accuracy_score(y_test, y_pred) * 100, "%")
print("Precision:", precision_score(y_test, y_pred, zero_division=0))
print("Recall:", recall_score(y_test, y_pred, zero_division=0))
print("F1-Score:", f1_score(y_test, y_pred, zero_division=0))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
```

```
plt.show()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Binary Cross-Entropy Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
```

INPUT & OUTPUT:



RESULT:

The binary classification neural network was successfully implemented using Binary Cross-Entropy loss and Sigmoid activation, and evaluated using accuracy, precision, recall, and F1-score.

Experiment 10: Comparison of SGD, RMSprop, and Adam Optimizers
Dataset: Fashion-MNIST Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to compare SGD, RMSprop, and Adam optimizers. Train identical neural network architectures and compare training loss, validation accuracy, and convergence performance.

Concept: Gradient-Based Optimization | CO2

AIM:

To compare SGD, RMSprop, and Adam optimizers using TensorFlow/Keras on the Fashion-MNIST dataset by evaluating training loss, validation accuracy, and convergence performance.

DATASET:

Dataset Classes:

Label	Fashion Item
0	**T-shirt/top**
1	**Trouser**
2	**Pullover**
3	**Dress**
4	**Coat**
5	**Sandal**
6	**Shirt**
7	**Sneaker**
8	**Bag**
9	**Ankle boot**

PROCEDURE:

- Import the required libraries.
- Load the Fashion-MNIST dataset.
- Normalize image pixel values between 0 and 1.
- Build an identical neural network architecture for all three optimizers.
- Compile three models using SGD, RMSprop, and Adam.
- Train all models using the same dataset and training settings.
- Record training loss and validation accuracy.
- Compare optimizer performance using graphs.
- Identify the optimizer with better convergence performance.

PROGRAM CODE:

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress',
               'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
print("Training Dataset:", x_train.shape)
print("Testing Dataset:", x_test.shape)
def create_model(optimizer):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])

    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
optimizers = {
    'SGD': tf.keras.optimizers.SGD(learning_rate=0.01),
```

```
    'RMSprop': tf.keras.optimizers.RMSprop(learning_rate=0.001),
    'Adam': tf.keras.optimizers.Adam(learning_rate=0.001)
}
results = {}
for name, optimizer in optimizers.items():
    tf.keras.utils.set_random_seed(42
    model = create_model(optimizer)
    history = model.fit(
        x_train, y_train,
        epochs=10,
        batch_size=64,
        validation_split=0.2,
        verbose=0
    )
    results[name] = history
    print("\nOptimizer:", name)
    print("Training Loss:", round(history.history['loss'][-1], 4))
    print("Validation Accuracy:",
          round(history.history['val_accuracy'][-1] * 100, 2), "%")
for name in results:
    plt.plot(results[name].history['loss'], label=name)
plt.title("Training Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
for name in results:
    plt.plot(results[name].history['val_accuracy'], label=name)
```

```
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
print("\nOptimizer Comparison")
for name in results:
    print(name,
        "Training Loss:", round(results[name].history['loss'][-1], 4),
        "Validation Accuracy:",
        round(results[name].history['val_accuracy'][-1] * 100, 2), "%")
```

INPUT & OUTPUT:

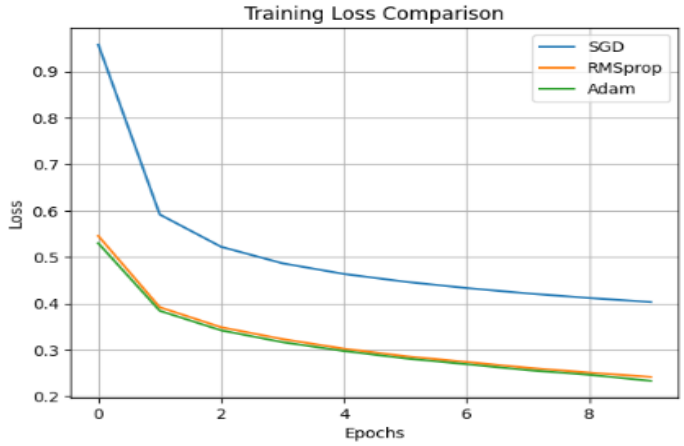


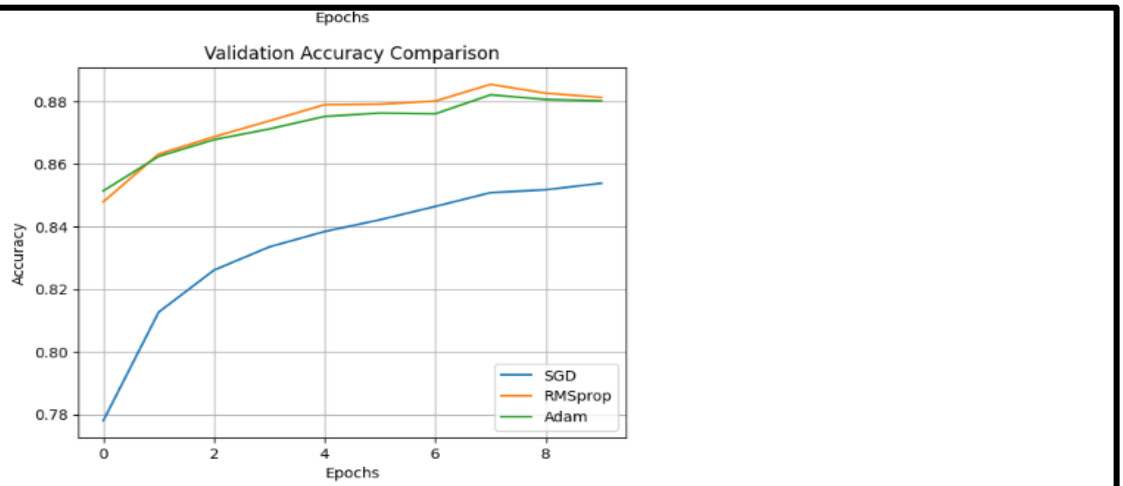
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Dataset: (60000, 28, 28)
Testing Dataset: (10000, 28, 28)

Optimizer: SGD
Training Loss: 0.4033
Validation Accuracy: 85.38 %

Optimizer: RMSprop
Training Loss: 0.2419
Validation Accuracy: 88.13 %

Optimizer: Adam
Training Loss: 0.2334
Validation Accuracy: 88.02 %
```






```
Optimizer Comparison
SGD Training Loss: 0.4033 Validation Accuracy: 85.38 %
RMSprop Training Loss: 0.2419 Validation Accuracy: 88.13 %
Adam Training Loss: 0.2334 Validation Accuracy: 88.02 %
```

RESULT:

SGD, RMSprop, and Adam optimizers were successfully implemented and compared using Fashion-MNIST. Their training loss, validation accuracy, and convergence performance were analyzed.

Phase 2: Deep Neural Networks and Regularization

Experiment 11: Deep Feedforward Network for Handwritten Digit Classification
Dataset: MNIST Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to develop a deep feedforward neural network for handwritten digit classification using the MNIST Dataset. Preprocess images, construct multiple hidden layers, train the model, and evaluate test accuracy.

Concept: Deep Neural Networks | CO2

AIM:

To develop a Deep Feedforward Neural Network using TensorFlow/Keras to classify handwritten digits from the MNIST dataset and evaluate its test accuracy.

DATASET:

Dataset Classes:

Label	Digit
0	**Zero**
1	**One**
2	**Two**
3	**Three**
4	**Four**
5	**Five**
6	**Six**
7	**Seven**
8	**Eight**
9	**Nine**

PROCEDURE:

- Import TensorFlow, NumPy, and Matplotlib.
- Load the MNIST dataset.
- Normalize image pixel values between 0 and 1.
- Flatten the 28 × 28 images into 784 input features.
- Build a deep neural network with multiple hidden layers.
- Compile the model using Adam optimizer and Sparse Categorical Crossentropy.
- Train the model using training images.

- Predict handwritten digits using test images.
- Evaluate and display test accuracy.

PROGRAM CODE:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
print("Training Data Shape:", x_train.shape)
print("Testing Data Shape:", x_test.shape)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train, y_train,
    epochs=10,
```

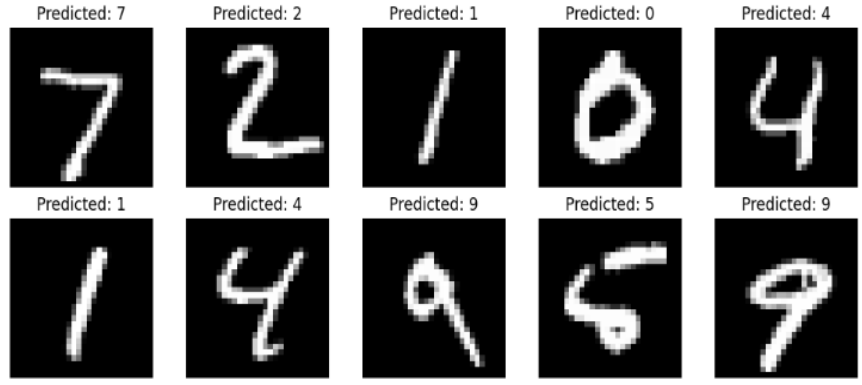
```
    batch_size=128,
    validation_split=0.2,
    verbose=1
)
test_loss, test_accuracy = model.evaluate(
    x_test, y_test, verbose=0
)
print("\nTest Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
predictions = model.predict(x_test[:10], verbose=0)
predicted_labels = np.argmax(predictions, axis=1)
print("\nActual Labels:   ", y_test[:10])
print("Predicted Labels:", predicted_labels)
plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Predicted: " + str(predicted_labels[i]))
    plt.axis('off')
plt.tight_layout()
plt.show()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

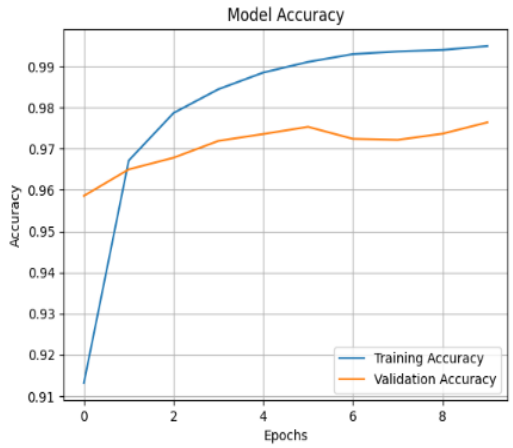
INPUT & OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training Data Shape: (60000, 28, 28)
Testing Data Shape: (10000, 28, 28)
Epoch 1/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - accuracy: 0.9131 - loss: 0.3037 - val_accuracy: 0.9586 - val_loss: 0.1382
Epoch 2/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9671 - loss: 0.1127 - val_accuracy: 0.9650 - val_loss: 0.1126
Epoch 3/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9787 - loss: 0.0715 - val_accuracy: 0.9678 - val_loss: 0.1037
Epoch 4/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.9845 - loss: 0.0515 - val_accuracy: 0.9719 - val_loss: 0.0957
Epoch 5/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9885 - loss: 0.0381 - val_accuracy: 0.9736 - val_loss: 0.1029
Epoch 6/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - accuracy: 0.9911 - loss: 0.0299 - val_accuracy: 0.9753 - val_loss: 0.0957
Epoch 7/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 5s 12ms/step - accuracy: 0.9930 - loss: 0.0229 - val_accuracy: 0.9724 - val_loss: 0.1083
Epoch 8/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9936 - loss: 0.0192 - val_accuracy: 0.9722 - val_loss: 0.1135
Epoch 9/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9940 - loss: 0.0178 - val_accuracy: 0.9737 - val_loss: 0.1167
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9949 - loss: 0.0158 - val_accuracy: 0.9764 - val_loss: 0.1076

Test Loss: 0.0904
Test Accuracy: 97.91 %
```

```
Actual Labels:    [7 2 1 0 4 1 4 9 5 9]
Predicted Labels: [7 2 1 0 4 1 4 9 5 9]
```





RESULT:

The Deep Feedforward Neural Network was successfully implemented using TensorFlow/Keras to classify handwritten digits from the MNIST dataset and evaluate test accuracy.

Experiment 12: Dropout to Reduce Overfitting in a Neural Network
Dataset: Fashion-MNIST Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to apply Dropout regularization. Train models with and without dropout and compare training accuracy, validation accuracy, and loss.

Concept: Dropout Regularization | CO2

AIM:

To implement Dropout regularization using TensorFlow/Keras on the Fashion-MNIST dataset and compare neural network performance with and without dropout.

DATASET:

Dataset Classes:

Label	Fashion Item
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

PROCEDURE:

- Import the required libraries.
- Load the Fashion-MNIST dataset.
- Normalize image pixel values between 0 and 1.
- Build two identical neural network models.
- Train the first model without dropout.
- Train the second model with Dropout layers.
- Compare training accuracy and validation accuracy.
- Compare training loss and validation loss using graphs.
- Analyze how dropout helps reduce overfitting.

PROGRAM CODE:

```
import tensorflow as tf
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
def create_model(dropout=False):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu')
    ])
    if dropout:
        model.add(tf.keras.layers.Dropout(0.5))
    model.add(tf.keras.layers.Dense(64, activation='relu'))
    if dropout:
        model.add(tf.keras.layers.Dropout(0.3))
    model.add(tf.keras.layers.Dense(10, activation='softmax'))
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
tf.keras.utils.set_random_seed(42)
model1 = create_model(dropout=False)
history1 = model1.fit(
    x_train, y_train,
    epochs=10,
```

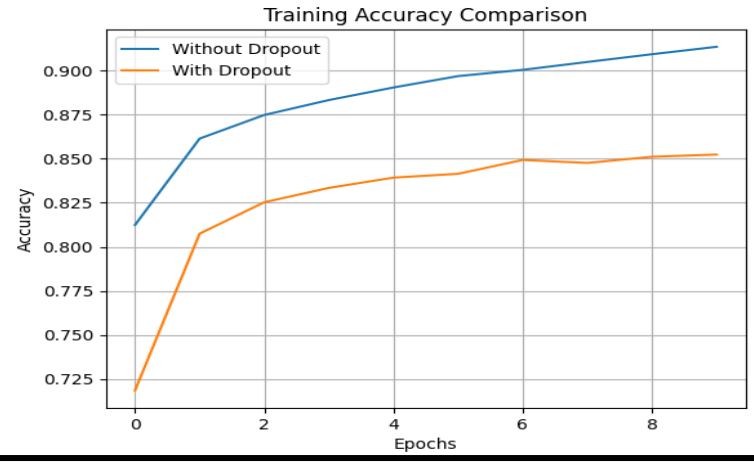
```
    batch_size=64,
    validation_split=0.2,
    verbose=0
)
tf.keras.utils.set_random_seed(42)
model2 = create_model(dropout=True)
history2 = model2.fit(
    x_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2,
    verbose=0
)
print("WITHOUT DROPOUT")
print("Training Accuracy:",
      round(history1.history['accuracy'][-1] * 100, 2), "%")
print("Validation Accuracy:",
      round(history1.history['val_accuracy'][-1] * 100, 2), "%")
print("Training Loss:",
      round(history1.history['loss'][-1], 4))
print("\nWITH DROPOUT")
print("Training Accuracy:",
      round(history2.history['accuracy'][-1] * 100, 2), "%")
print("Validation Accuracy:",
      round(history2.history['val_accuracy'][-1] * 100, 2), "%")
print("Training Loss:",
      round(history2.history['loss'][-1], 4))
plt.plot(history1.history['accuracy'], label='Without Dropout')
plt.plot(history2.history['accuracy'], label='With Dropout')
```

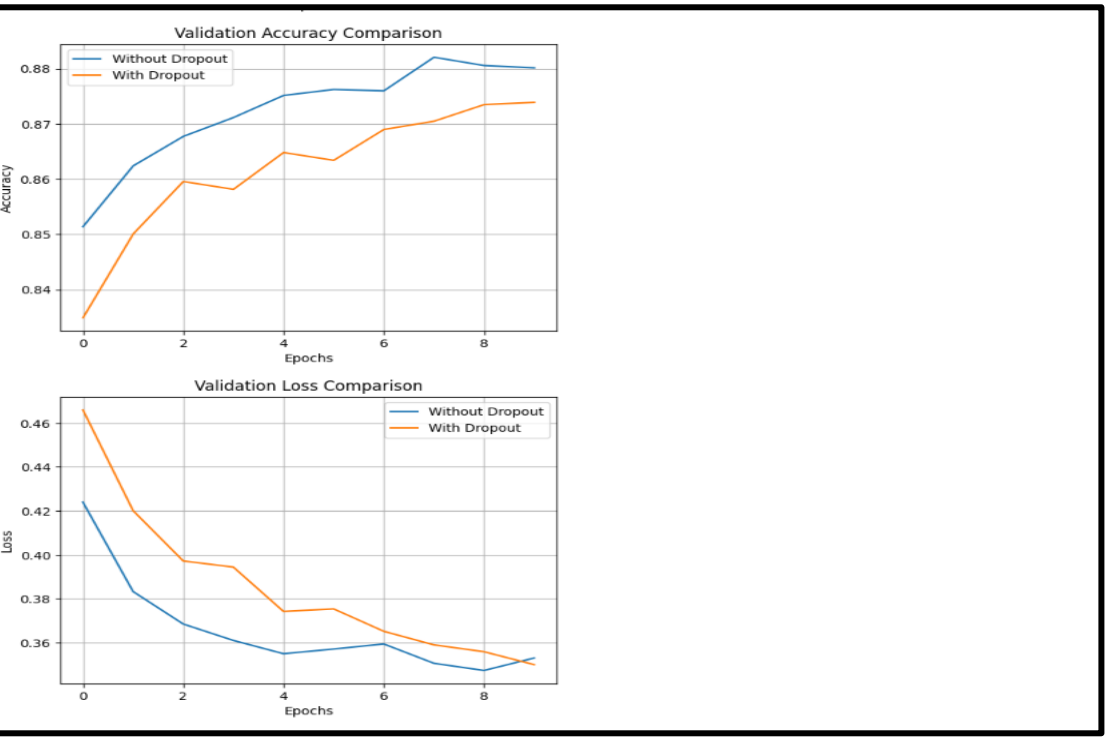
```
plt.title("Training Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
plt.plot(history1.history['val_accuracy'], label='Without Dropout')
plt.plot(history2.history['val_accuracy'], label='With Dropout')
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
plt.plot(history1.history['val_loss'], label='Without Dropout')
plt.plot(history2.history['val_loss'], label='With Dropout')
plt.title("Validation Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
```

INPUT & OUTPUT:

```
WITHOUT DROPOUT
Training Accuracy: 91.36 %
Validation Accuracy: 88.02 %
Training Loss: 0.2334

WITH DROPOUT
Training Accuracy: 85.24 %
Validation Accuracy: 87.39 %
Training Loss: 0.4046
```





RESULT:

Dropout regularization was successfully implemented and compared with a model without dropout. Its effectiveness in reducing overfitting was analyzed using accuracy and loss graphs.

Experiment 13: L2 Regularization in a Multilayer Neural Network
Dataset: Breast Cancer Wisconsin Diagnostic Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to implement L2 regularization in a multilayer neural network. Apply L2 penalties to network weights and analyse the effect on model performance.

Concept: Weight Regularization | CO2

AIM:

To implement L2 regularization in a multilayer neural network using TensorFlow/Keras on the Breast Cancer Wisconsin dataset and analyze its effect on model performance.

DATASET:

Sample Dataset:

Mean Radius	Mean Texture	Mean Perimeter	Mean Area	Diagnosis
17.99	10.38	122.80	1001.0	Malignant
20.57	17.77	132.90	1326.0	Malignant
19.69	21.25	130.00	1203.0	Malignant
11.42	20.38	77.58	386.1	Benign
20.29	14.34	135.10	1297.0	Malignant

PROCEDURE:

- Import the required libraries.
- Load the Breast Cancer Wisconsin dataset.
- Separate input features and target labels.
- Split the dataset into training and testing sets.
- Normalize the input features using StandardScaler.
- Build two neural networks with identical architectures.
- Apply L2 regularization to the weights of the second model.
- Train both models using the same optimizer and settings.
- Compare accuracy and loss values.
- Analyze the effect of L2 regularization.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
def create_model(use_l2=False):
    regularizer = tf.keras.regularizers.l2(0.01) if use_l2 else None
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(30,)),
        tf.keras.layers.Dense(16, activation='relu',
                              kernel_regularizer=regularizer),
        tf.keras.layers.Dense(8, activation='relu',
                              kernel_regularizer=regularizer),
        tf.keras.layers.Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer='adam',
```

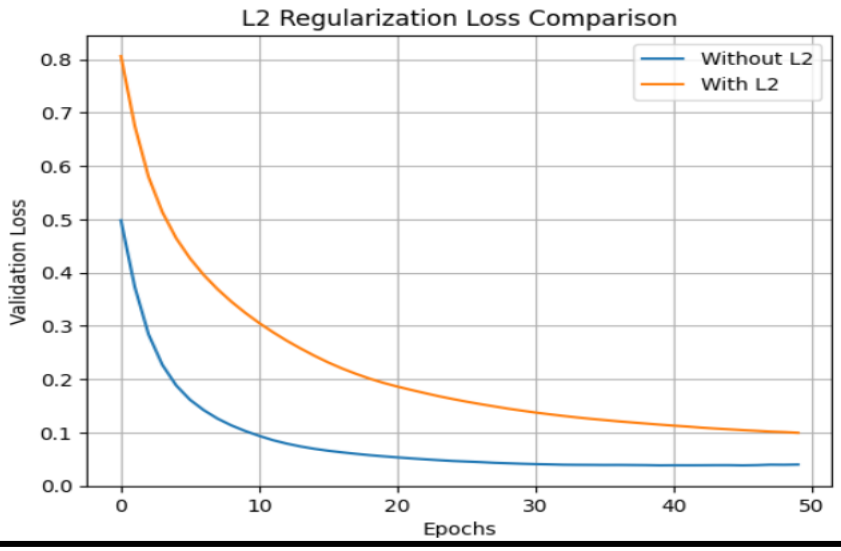
```
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model
tf.keras.utils.set_random_seed(42)
model1 = create_model(False)
history1 = model1.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)
tf.keras.utils.set_random_seed(42)
model2 = create_model(True)
history2 = model2.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.2,
    verbose=0
)
pred1 = (model1.predict(X_test, verbose=0) >= 0.5).astype(int)
pred2 = (model2.predict(X_test, verbose=0) >= 0.5).astype(int)
print("WITHOUT L2 REGULARIZATION")
print("Test Accuracy:",
      accuracy_score(y_test, pred1) * 100, "%")
print("Training Loss:", history1.history['loss'][-1])
print("Validation Loss:", history1.history['val_loss'][-1])
```

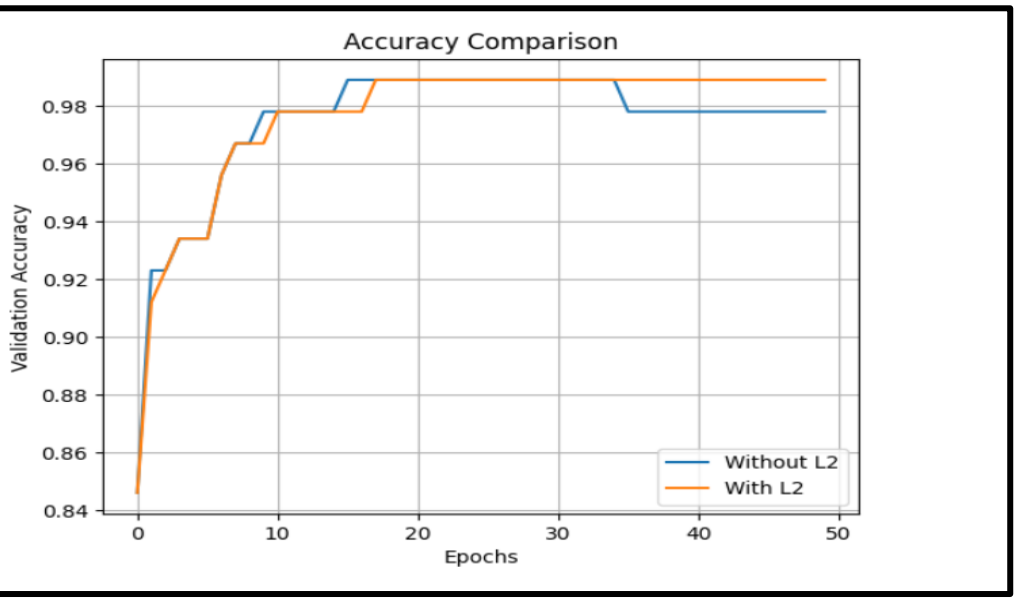
```
print("\nWITH L2 REGULARIZATION")
print("Test Accuracy:",
    accuracy_score(y_test, pred2) * 100, "%")
print("Training Loss:", history2.history['loss'][-1])
print("Validation Loss:", history2.history['val_loss'][-1])
plt.plot(history1.history['val_loss'], label='Without L2')
plt.plot(history2.history['val_loss'], label='With L2')
plt.title("L2 Regularization Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid()
plt.show()
plt.plot(history1.history['val_accuracy'], label='Without L2')
plt.plot(history2.history['val_accuracy'], label='With L2')
plt.title("Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid()
plt.show()
```

INPUT & OUTPUT:

```
WITHOUT L2 REGULARIZATION
Test Accuracy: 97.36842105263158 %
Training Loss: 0.02144465036690235
Validation Loss: 0.03966893255710602

WITH L2 REGULARIZATION
Test Accuracy: 97.36842105263158 %
Training Loss: 0.10069318115711212
Validation Loss: 0.09925827383995056
```





RESULT:

L2 regularization was successfully implemented in a multilayer neural network, and its effect on model accuracy and loss was analyzed.

Experiment 14: Neural Network Performance With and Without Regularization
Dataset: Fashion-MNIST Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to compare neural networks without regularization, with Dropout, and with L2 regularization. Evaluate training and testing performance and visualize the differences in generalization.

Concept: Model Generalization | CO2

AIM:

To compare neural network performance without regularization, with Dropout, and with L2 regularization using the Fashion-MNIST dataset.

PROCEDURE:

1. **Import TensorFlow, Keras, NumPy and Matplotlib.**
2. **Load the Fashion-MNIST dataset.**
3. **Normalize the image pixel values.**
4. **Build three neural network models:**

- **Without regularization.**
- **With Dropout regularization.**
- **With L2 regularization.**

1. **Train all three models using the same settings.**
2. **Evaluate training and testing accuracy.**
3. **Compare the results using graphs.**

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras import layers, models, regularizers

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train / 255.0

x_test = x_test / 255.0

def create_model(reg_type=None):

    model = models.Sequential()

    model.add(layers.Flatten(input_shape=(28, 28))
```

```
    if reg_type == "l2":
        model.add(layers.Dense(
            128, activation='relu',
            kernel_regularizer=regularizers.l2(0.001)))
        model.add(layers.Dense(
            64, activation='relu',
            kernel_regularizer=regularizers.l2(0.001)))
    else:
        model.add(layers.Dense(128, activation='relu'))
        if reg_type == "dropout":
            model.add(layers.Dropout(0.5))
        model.add(layers.Dense(64, activation='relu'))
        if reg_type == "dropout":
            model.add(layers.Dropout(0.3))
    model.add(layers.Dense(10, activation='softmax'))
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
model_types = {
    "Without Regularization": None,
    "Dropout": "dropout",
    "L2 Regularization": "l2"
}
results = {}
for name, reg in model_types.items():
    print("\nTraining:", name)
```

```
    model = create_model(reg
    history = model.fit(
        x_train, y_train,
        epochs=10,
        batch_size=128,
        validation_split=0.2,
        verbose=1
    )
    train_loss, train_acc = model.evaluate(
        x_train, y_train, verbose=0)
    test_loss, test_acc = model.evaluate(
        x_test, y_test, verbose=0)
    results[name] = {
        "Train Accuracy": train_acc,
        "Test Accuracy": test_acc,
        "Train Loss": train_loss,
        "Test Loss": test_loss,
        "History": history
    }
    print("Training Accuracy:", round(train_acc * 100, 2), "%")
    print("Testing Accuracy:", round(test_acc * 100, 2), "%")
    print("Training Loss:", round(train_loss, 4))
    print("Testing Loss:", round(test_loss, 4))
names = list(results.keys())
train_acc = [results[n]["Train Accuracy"] * 100 for n in names]
test_acc = [results[n]["Test Accuracy"] * 100 for n in names]
plt.figure(figsize=(9, 5))
x = np.arange(len(names))
width = 0.35
```

```
plt.bar(x - width/2, train_acc, width, label='Training Accuracy')
plt.bar(x + width/2, test_acc, width, label='Testing Accuracy')
plt.xticks(x, names, rotation=10)
plt.ylabel("Accuracy (%)")
plt.title("Performance With and Without Regularization")
plt.legend()
plt.tight_layout()
plt.show()
plt.figure(figsize=(9, 5))
for name in names:
    history = results[name]["History"]
    plt.plot(history.history['val_accuracy'], label=name)
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Model Generalization Comparison")
plt.legend()
plt.show()
```

INPUT:

Fashion-MNIST Dataset:

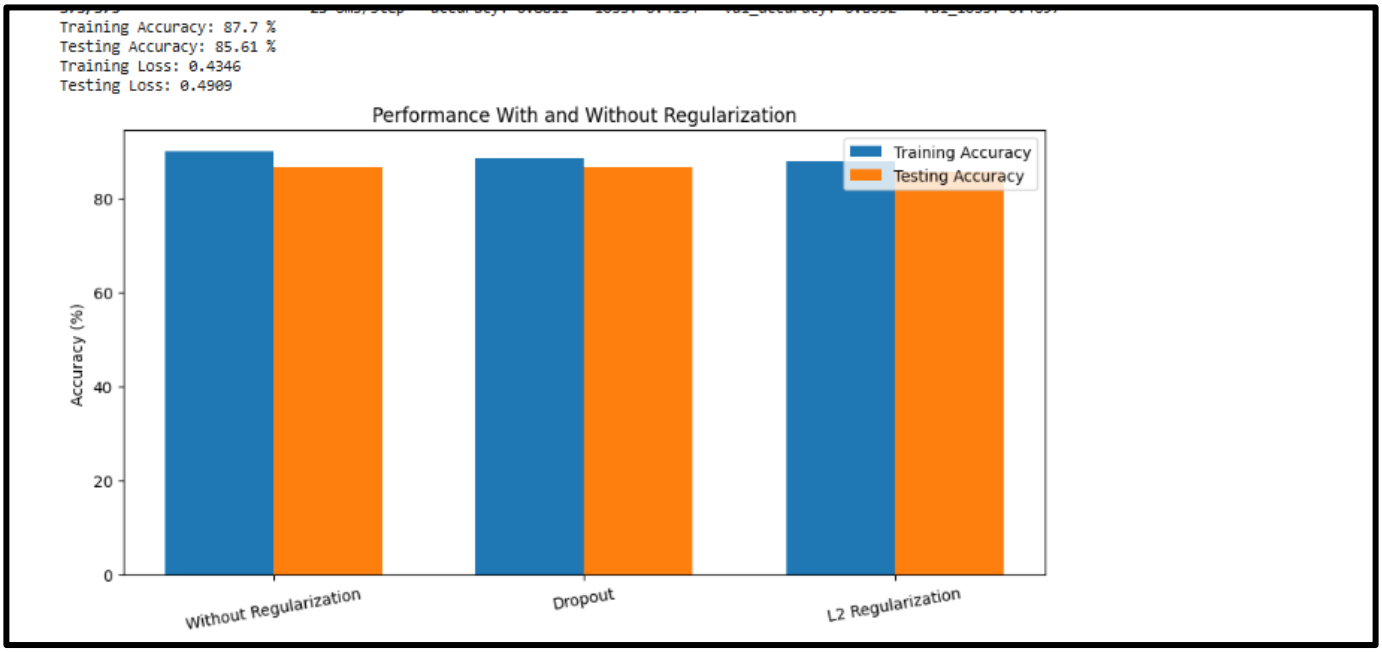
- 60,000 training images.
- 10,000 testing images.
- Image size: 28 × 28 pixels.
- 10 clothing categories.
- Pixel values normalized between 0 and 1.

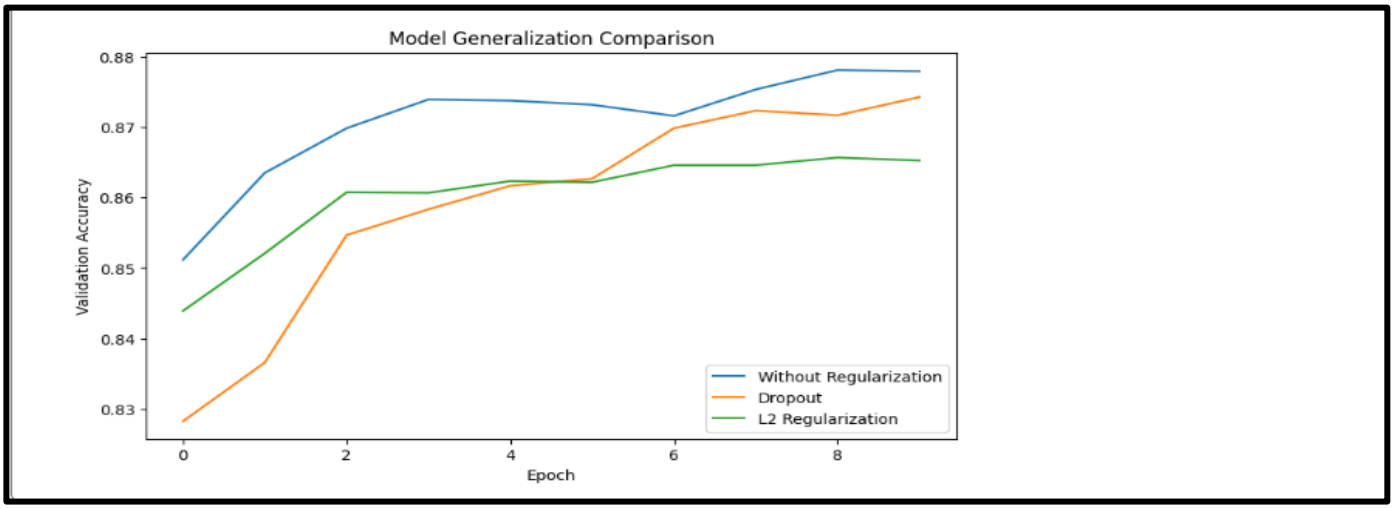
Regularization methods:

1. Without regularization.
2. Dropout (0.5 and 0.3).
3. L2 regularization (0.001).

OUTPUT:

```
Training Accuracy: 87.7 %
Testing Accuracy: 85.61 %
Training Loss: 0.4346
Testing Loss: 0.4909
```





RESULT:

Successfully compared neural networks with and without regularization and analyzed their performance and generalization using Fashion-MNIST.

Phase 3: Convolutional Neural Networks

Experiment 15: CNN for Handwritten Digit Classification Using MNIST
Dataset: MNIST Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to develop a CNN for handwritten digit classification. Implement convolutional, pooling, flattening, and dense layers, and evaluate the model using test accuracy.

Concept: Convolutional Neural Networks | CO2

AIM:

To develop a Convolutional Neural Network (CNN) using TensorFlow/Keras to classify handwritten digits from 0 to 9 and evaluate its test accuracy.

PROCEDURE:

- Import TensorFlow, Keras and Matplotlib.
- Load the MNIST handwritten digit dataset.
- Normalize pixel values between 0 and 1.
- Reshape images for CNN input.
- Build a CNN using convolutional, pooling, flattening and dense layers.
- Train the model using training images.
- Evaluate the model using testing images.
- Display test accuracy and sample predictions.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train / 255.0

x_test = x_test / 255.0

x_train = x_train.reshape(-1, 28, 28, 1)

x_test = x_test.reshape(-1, 28, 28, 1)

model = tf.keras.Sequential([

    tf.keras.layers.Input(shape=(28, 28, 1)),

    tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),

    tf.keras.layers.MaxPooling2D((2, 2)),
```

```
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.summary()
history = model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Loss:", test_loss)
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
predictions = model.predict(x_test[:5])
for i in range(5):
    print("Actual Digit:", y_test[i])
    print("Predicted Digit:", np.argmax(predictions[i]))
plt.figure(figsize=(10, 4))
for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
```

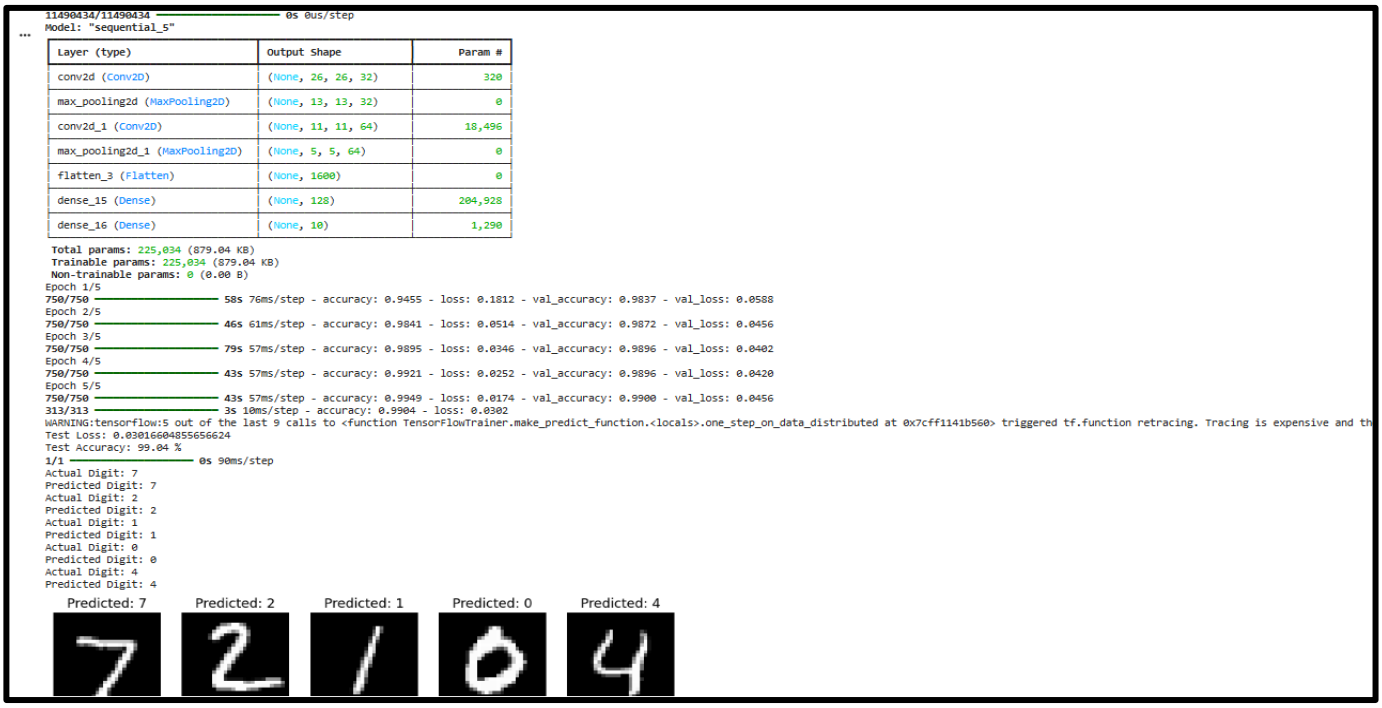
```
    plt.title("Predicted: " + str(np.argmax(predictions[i])))
    plt.axis('off')
plt.show()
```

INPUT:

MNIST Dataset:

- 60,000 training images.
- 10,000 testing images.
- Image size: 28 × 28 pixels.
- Grayscale handwritten digit images.
- 10 classes: digits 0 to 9.
- Pixel values normalized between 0 and 1.

OUTPUT:



RESULT:

Successfully developed a CNN model to classify handwritten digits using the MNIST dataset and evaluated its test accuracy.

Experiment 16: Convolution and Pooling Operations on an Image
Dataset: CIFAR-10 Dataset

Python Programming Question: Write a Python program using OpenCV and NumPy to perform convolution and pooling operations on sample images. Apply edge detection and sharpening filters, implement max pooling and average pooling, and display the processed images.

Concept: Convolution and Pooling | CO2

AIM:

To perform convolution and pooling operations on CIFAR-10 images using OpenCV and NumPy. Apply edge detection, sharpening, max pooling and average pooling, and display the processed images.

PROCEDURE:

- Import OpenCV, NumPy and Matplotlib.
- Load the CIFAR-10 dataset.
- Select a sample image.
- Convert the image into grayscale.
- Apply edge detection using a Sobel filter.
- Apply a sharpening filter.
- Perform max pooling and average pooling.
- Display all processed images.

PROGRAM CODE:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

image = x_train[0]

gray = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

edges = cv2.magnitude(sobel_x, sobel_y)

edges = cv2.normalize(edges, None, 0, 255,

                      cv2.NORM_MINMAX).astype(np.uint8)

kernel = np.array([
```

```
    [0, -1, 0],
    [-1, 5, -1],
    [0, -1, 0]
])
sharpened = cv2.filter2D(gray, -1, kernel)
max_pool = cv2.resize(
    gray[::2, ::2], (15, 15),
    interpolation=cv2.INTER_NEAREST
)
average_pool = cv2.blur(gray, (2, 2))[::2, ::2]
plt.figure(figsize=(12, 7))
images = [
    (image, "Original Image"),
    (gray, "Grayscale Image"),
    (edges, "Edge Detection"),
    (sharpened, "Sharpened Image"),
    (max_pool, "Max Pooling"),
    (average_pool, "Average Pooling")
]
for i, (img, title) in enumerate(images):
    plt.subplot(2, 3, i + 1)
    if len(img.shape) == 3:
        plt.imshow(img)
    else:
        plt.imshow(img, cmap='gray')
    plt.title(title)
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
print("Convolution and Pooling Operations Completed")
print("Original Image Shape:", image.shape)
print("Max Pooling Shape:", max_pool.shape)
print("Average Pooling Shape:", average_pool.shape)
```

INPUT:

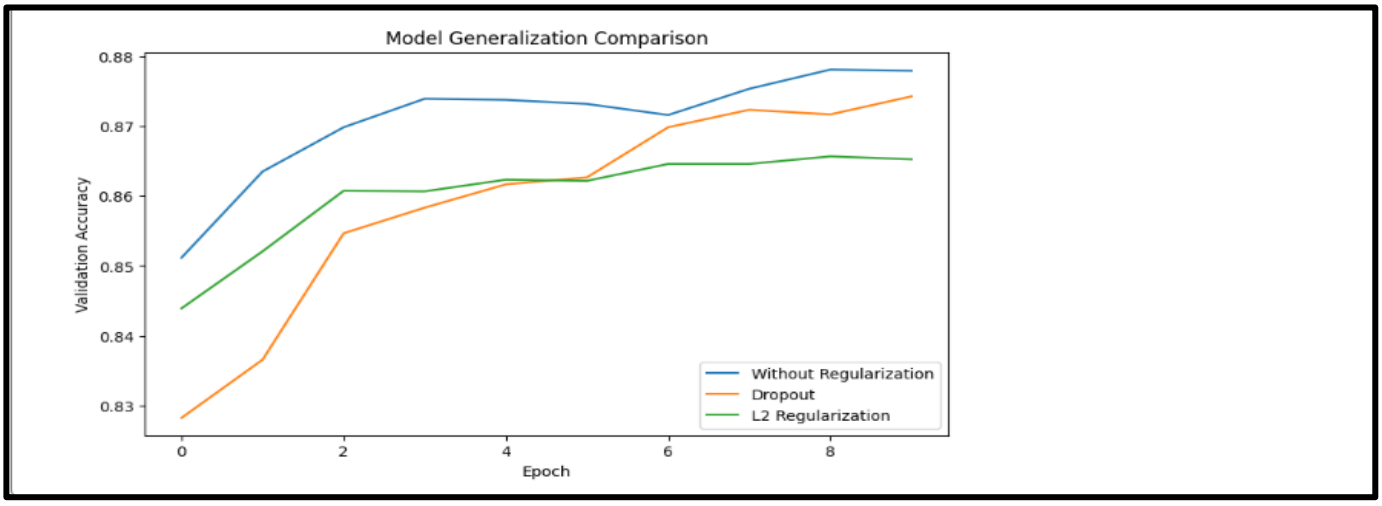
CIFAR-10 Dataset:

- 50,000 training images.
- 10,000 testing images.
- Image size: 32 × 32 pixels.
- RGB color images.
- 10 object categories.

Sample input:

- Select the first image from the CIFAR-10 training dataset.
- Convert the RGB image into grayscale.
- Apply edge detection and sharpening filters.
- Perform max pooling and average pooling.

OUTPUT:



RESULT:

Successfully performed convolution, edge detection, sharpening, max pooling and average pooling on CIFAR-10 images using OpenCV and NumPy.

Experiment 17: Comparison of Max Pooling and Average Pooling in CNN
Dataset: CIFAR-10 Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to compare Max Pooling and Average Pooling. Develop two CNN models with identical architectures except for the pooling operation, and compare accuracy and loss.

Concept: Pooling Operations | CO2

AIM:

To compare Max Pooling and Average Pooling in CNN using TensorFlow/Keras and evaluate their accuracy and loss on the CIFAR-10 dataset.

PROCEDURE:

- Import TensorFlow, Keras, NumPy and Matplotlib.
- Load the CIFAR-10 dataset.
- Normalize image pixel values between 0 and 1.
- Create two CNN models with identical architectures.
- Apply Max Pooling in the first model.
- Apply Average Pooling in the second model.
- Train both models using the same settings.
- Compare their test accuracy and loss using graphs.

PROGRAM CODE:

```
import tensorflow as tf

import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0

def create_model(pooling_type):

    model = tf.keras.Sequential([

        tf.keras.layers.Input(shape=(32, 32, 3)),

        tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),

        tf.keras.layers.BatchNormalization(),

        pooling_type((2, 2))

        tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),

        tf.keras.layers.BatchNormalization(),
```

```
        pooling_type((2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
max_model = create_model(tf.keras.layers.MaxPooling2D)
avg_model = create_model(tf.keras.layers.AveragePooling2D)
print("Training Max Pooling Model")
max_history = max_model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
max_loss, max_accuracy = max_model.evaluate(x_test, y_test, verbose=0)
print("\nTraining Average Pooling Model")
avg_history = avg_model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
```

```
)
avg_loss, avg_accuracy = avg_model.evaluate(x_test, y_test, verbose=0)
print("\nMax Pooling Results")
print("Test Accuracy:", round(max_accuracy * 100, 2), "%")
print("Test Loss:", round(max_loss, 4))
print("\nAverage Pooling Results")
print("Test Accuracy:", round(avg_accuracy * 100, 2), "%")
print("Test Loss:", round(avg_loss, 4))
plt.figure(figsize=(8, 5))
plt.bar(
    ['Max Pooling', 'Average Pooling'],
    [max_accuracy * 100, avg_accuracy * 100]
)
plt.ylabel("Test Accuracy (%)")
plt.title("Max Pooling vs Average Pooling")
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(max_history.history['val_loss'], label='Max Pooling')
plt.plot(avg_history.history['val_loss'], label='Average Pooling')
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Validation Loss Comparison")
plt.legend()
plt.show()
```

INPUT:

CIFAR-10 Dataset:

- 50,000 training images.
- 10,000 testing images.
- Image size: 32 × 32 pixels.
- RGB color images.
- 10 classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

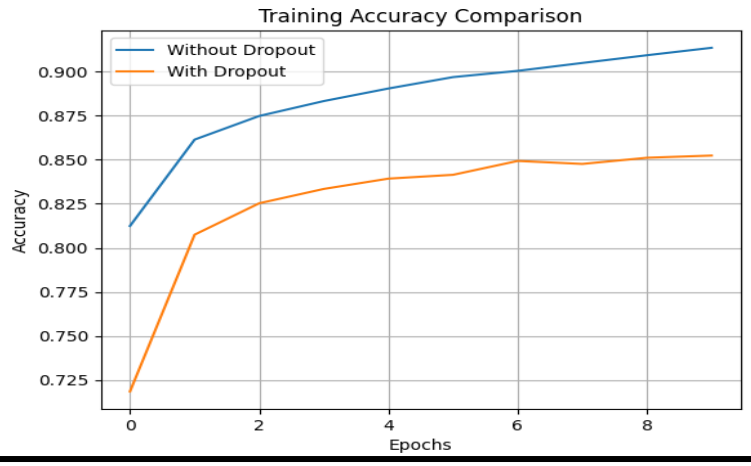
Pooling operations:

1. Max Pooling – selects the maximum value from each region.
2. Average Pooling – calculates the average value from each region.

OUTPUT:

```
WITHOUT DROPOUT
Training Accuracy: 91.36 %
Validation Accuracy: 88.02 %
Training Loss: 0.2334

WITH DROPOUT
Training Accuracy: 85.24 %
Validation Accuracy: 87.39 %
Training Loss: 0.4046
```



RESULT:

Successfully compared Max Pooling and Average Pooling CNN models using CIFAR-10 and analyzed their accuracy and loss.

Experiment 18: CNN for Cats and Dogs Classification
Dataset: Kaggle Dogs vs. Cats Dataset
Python Programming Question: Write a Python program using TensorFlow/Keras to develop a CNN for classifying images of cats and dogs. Perform image resizing, normalization, training, prediction, and evaluation using test images.

Concept: Image Classification | CO2

AIM:

To develop a Convolutional Neural Network (CNN) using TensorFlow/Keras to classify cat and dog images through image resizing, normalization, training, prediction and testing.

PROCEDURE:

- Import TensorFlow, Keras, NumPy and Matplotlib.
- Load the Kaggle Dogs vs. Cats dataset.
- Resize images to 150 × 150 pixels.
- Normalize image pixel values between 0 and 1.
- Divide the labeled images into training and testing sets.
- Build a CNN using convolutional, pooling and dense layers.
- Train the CNN model.
- Evaluate the model using test images.
- Predict whether the images contain cats or dogs.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

import os

import glob

from sklearn.model_selection import train_test_split

DATASET_PATH = "/content/dogs-vs-cats/train"

cat_files = glob.glob(os.path.join(DATASET_PATH, "cat.*.jpg"))

dog_files = glob.glob(os.path.join(DATASET_PATH, "dog.*.jpg"))

files = cat_files + dog_files

labels = [0] * len(cat_files) + [1] * len(dog_files)
```

```
print("Total Images:", len(files))
train_files, test_files, train_labels, test_labels = train_test_split(
    files, labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)
def preprocess_image(path, label):
    image = tf.io.read_file(path)
    image = tf.io.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, (150, 150))
    image = image / 255.0
    return image, label
train_data = tf.data.Dataset.from_tensor_slices(
    (train_files, train_labels)
)
test_data = tf.data.Dataset.from_tensor_slices(
    (test_files, test_labels)
)
train_data = train_data.map(
    preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE
)
test_data = test_data.map(
    preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE
)
train_data = train_data.shuffle(1000).batch(32).prefetch(
    tf.data.AUTOTUNE
```

```
)
test_data = test_data.batch(32).prefetch(
    tf.data.AUTOTUNE
)model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(150, 150, 3)),
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Conv2D(128, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2, 2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()
history = model.fit(
    train_data,
    epochs=5,
    validation_data=test_data
)
test_loss, test_accuracy = model.evaluate(test_data)
print("\nTest Accuracy:", round(test_accuracy * 100, 2), "%")
```

```
print("Test Loss:", round(test_loss, 4))
for images, labels in test_data.take(1):
    predictions = model.predict(images)
    for i in range(5):
        predicted_class = (
            "Dog" if predictions[i][0] >= 0.5 else "Cat"
        )
        actual_class = (
            "Dog" if labels[i].numpy() == 1 else "Cat”
        print("Actual:", actual_class)
        print("Predicted:", predicted_class)
        plt.imshow(images[i].numpy())
        plt.title("Predicted: " + predicted_class)
        plt.axis('off')
        plt.show()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Testing Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Cats and Dogs Classification")
plt.legend()
plt.show()
```

INPUT:

Kaggle Dogs vs. Cats Dataset:

- Contains labeled cat and dog images.
- Two categories: Cats and Dogs.
- Images are resized to 150 × 150 pixels.
- RGB color images.

- Pixel values normalized between 0 and 1.
- 80% images used for training.
- 20% images used for testing.

Sample input:

- Cat image → Class 0
- Dog image → Class 1

OUTPUT:

```
        output = sigmoid(np.dot(h, W2) + b2)
        prediction = 1 if output[0][0] >= 0.5 else 0

        print(X[i], " | ", y[i][0], " | ", prediction)
```

```
Input | Actual | Predicted
[0 0]  |  0  |  0
[0 1]  |  1  |  1
[1 0]  |  1  |  1
[1 1]  |  0  |  0
```

RESULT:

Successfully developed a CNN model to classify cat and dog images and evaluated its performance using test images.

Experiment 19: CNN Hyperparameter Tuning Using Different Filter Sizes and Numbers of Filters
Dataset: CIFAR-10 Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to compare CNN architectures with different filter sizes (3 × 3 and 5 × 5) and numbers of filters (16, 32, and 64). Compare accuracy, loss, and training time.

Concept: CNN Hyperparameter Tuning | CO2

AIM:

To compare CNN architectures using different filter sizes (3×3 and 5×5) and numbers of filters (16, 32 and 64) and evaluate accuracy, loss and training time.

PROCEDURE:

- Import TensorFlow, Keras, NumPy and Matplotlib.
- Load the CIFAR-10 dataset.
- Normalize image pixel values between 0 and 1.
- Create CNN models with different filter sizes and filter numbers.
- Train each model using the same dataset and settings.
- Record test accuracy, test loss and training time.
- Compare the results using graphs.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

import time

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0

def create_model(filter_size, filters):

    model = tf.keras.Sequential([

        tf.keras.layers.Input(shape=(32, 32, 3)),

        tf.keras.layers.Conv2D(

            filters, (filter_size, filter_size),

            activation='relu', padding='same'

        ),
```

```
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Conv2D(
            filters * 2, (filter_size, filter_size),
            activation='relu', padding='same'
        ),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
filter_sizes = [3, 5]
filter_numbers = [16, 32, 64]
results = []
for size in filter_sizes:
    for filters in filter_numbers:
        print("\nFilter Size:", size, "x", size)
        print("Number of Filters:", filters)
        model = create_model(size, filters)
        start_time = time.time()
        model.fit(
            x_train, y_train,
            epochs=3,
            batch_size=64,
```

```
            validation_split=0.2,
            verbose=1
        )
        training_time = time.time() - start_time
        test_loss, test_accuracy = model.evaluate(
            x_test, y_test, verbose=0
        )
        results.append([
            size, filters,
            test_accuracy * 100,
            test_loss,
            training_time
        ])
        print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
        print("Test Loss:", round(test_loss, 4))
        print("Training Time:", round(training_time, 2), "seconds")
print("\nHYPERPARAMETER COMPARISON")
print("Size  Filters  Accuracy  Loss  Time")
for r in results:
    print(r[0], "x", r[0], " ", r[1],
          " ", round(r[2], 2), "%",
          " ", round(r[3], 4),
          " ", round(r[4], 2), "sec")
labels = [
    f"{r[0]}x{r[0]}-{r[1]}" for r in results
]
accuracy = [r[2] for r in results]

plt.figure(figsize=(10, 5))
```

```
plt.bar(labels, accuracy)
plt.xlabel("Filter Size and Number of Filters")
plt.ylabel("Test Accuracy (%)")
plt.title("CNN Accuracy Comparison")
plt.show()
loss = [r[3] for r in results]
plt.figure(figsize=(10, 5))
plt.bar(labels, loss)
plt.xlabel("Filter Size and Number of Filters")
plt.ylabel("Test Loss")
plt.title("CNN Loss Comparison")
plt.show()
training_times = [r[4] for r in results]
plt.figure(figsize=(10, 5))
plt.bar(labels, training_times)
plt.xlabel("Filter Size and Number of Filters")
plt.ylabel("Training Time (seconds)")
plt.title("CNN Training Time Comparison")
plt.show()
```

INPUT:

CIFAR-10 Dataset:

- 50,000 training images.
- 10,000 testing images.
- Image size: 32 × 32 pixels.
- RGB color images.
- 10 object categories.

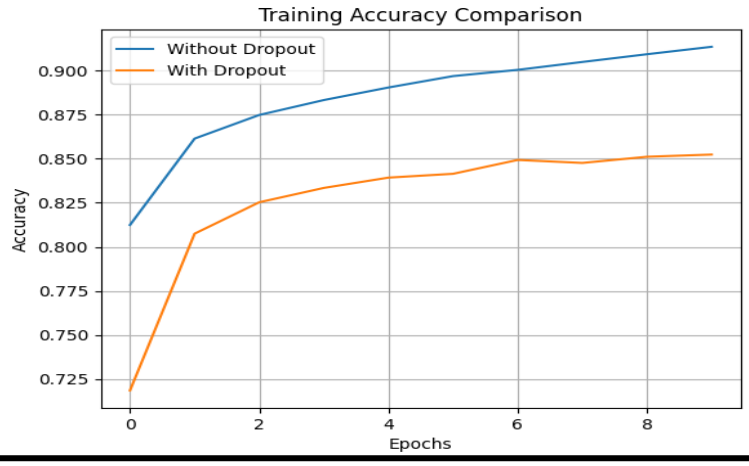
Hyperparameter combinations:

1. Filter size: 3×3, Number of filters: 16.
2. Filter size: 3×3, Number of filters: 32.
3. Filter size: 3×3, Number of filters: 64.
4. Filter size: 5×5, Number of filters: 16.
5. Filter size: 5×5, Number of filters: 32.
6. Filter size: 5×5, Number of filters: 64.

OUTPUT:

```
WITHOUT DROPOUT
Training Accuracy: 91.36 %
Validation Accuracy: 88.02 %
Training Loss: 0.2334

WITH DROPOUT
Training Accuracy: 85.24 %
Validation Accuracy: 87.39 %
Training Loss: 0.4046
```



RESULT:

Successfully compared six CNN architectures using different filter sizes and numbers of filters based on accuracy, loss and training time.

Experiment 20: CNN Performance Evaluation Using Classification Metrics
Dataset: CIFAR-10 Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras and Scikit-learn to evaluate a CNN using accuracy, precision, recall, F1-score, and a confusion matrix. Visualize the classification results.

Concept: CNN Performance Evaluation | CO2

AIM:

To evaluate a CNN model using accuracy, precision, recall, F1-score and confusion matrix with TensorFlow/Keras and Scikit-learn.

PROCEDURE:

- Import TensorFlow, Keras, NumPy, Matplotlib and Scikit-learn.
- Load the CIFAR-10 dataset.
- Normalize image pixel values between 0 and 1.
- Build a CNN model using convolutional and pooling layers.
- Train the model using training images.
- Predict the classes of testing images.
- Calculate accuracy, precision, recall and F1-score.
- Generate and visualize the confusion matrix.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import (

    accuracy_score,

    precision_score,

    recall_score,

    f1_score,

    confusion_matrix,

    classification_report

)

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()
```

```
y_train = y_train.flatten()
y_test = y_test.flatten()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
class_names = [
    'Airplane', 'Automobile', 'Bird', 'Cat', 'Deer',
    'Dog', 'Frog', 'Horse', 'Ship', 'Truck'
]
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(32, 32, 3)),
    tf.keras.layers.Conv2D(32, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
model.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2
)
```

```
predictions = model.predict(x_test)
y_pred = np.argmax(predictions, axis=1)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
    y_test, y_pred, average='macro', zero_division=0
)
recall = recall_score(
    y_test, y_pred, average='macro', zero_division=0
)
f1 = f1_score(
    y_test, y_pred, average='macro', zero_division=0
)
print("\nCNN PERFORMANCE EVALUATION")
print("Accuracy :", round(accuracy * 100, 2), "%")
print("Precision:", round(precision * 100, 2), "%")
print("Recall   :", round(recall * 100, 2), "%")
print("F1-Score :", round(f1 * 100, 2), "%")
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=class_names,
    zero_division=0
))
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
```

```
    cmap='Blues',
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("CNN Confusion Matrix")
plt.show()
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
values = [
    accuracy * 100,
    precision * 100,
    recall * 100,
    f1 * 100
]
plt.figure(figsize=(8, 5))
plt.bar(metrics, values)
plt.ylabel("Score (%)")
plt.title("CNN Classification Metrics")
plt.ylim(0, 100)
plt.show()
```

INPUT:

CIFAR-10 Dataset:

- 50,000 training images.
- 10,000 testing images.
- Image size: 32 × 32 pixels.
- RGB color images.
- 10 classes:

1. Airplane
2. Automobile
3. Bird
4. Cat
5. Deer
6. Dog
7. Frog
8. Horse
9. Ship
10. Truck

Evaluation Metrics:

- Accuracy – overall correct predictions.
- Precision – correctness of positive predictions.
- Recall – correctly identified actual samples.
- F1-score – balance between precision and recall.
- Confusion matrix – compares actual and predicted classes.

OUTPUT:

```
# Plot Accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Model Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()
```

```
Epoch 10/10
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9949 - loss: 0.0158 - val_accuracy: 0.9764 - val_loss: 0.1076

Test Loss: 0.0904
Test Accuracy: 97.91 %

Actual Labels:    [7 2 1 0 4 1 4 9 5 9]
Predicted Labels: [7 2 1 0 4 1 4 9 5 9]
```

RESULT:

Successfully evaluated the CNN model using accuracy, precision, recall, F1-score and confusion matrix on the CIFAR-10 dataset.

Experiment 21: Batch Normalization and Its Effect on Training Performance
Dataset: CIFAR-10 Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to implement batch normalization in a CNN. Train two models with and without batch normalization and compare training loss, validation accuracy, and convergence behaviour.

Concept: Regularization Techniques | CO2

AIM:

To implement Batch Normalization in a CNN using TensorFlow/Keras and compare training loss, validation accuracy and convergence behaviour with and without Batch Normalization.

PROCEDURE:

- Import TensorFlow, Keras, NumPy and Matplotlib.
- Load the CIFAR-10 dataset.
- Normalize image pixel values between 0 and 1.
- Build two identical CNN models.
- Add Batch Normalization layers to the second model.
- Train both models using the same settings.
- Compare training loss and validation accuracy.
- Plot graphs to analyze convergence behaviour.

PROGRAM CODE:

```
import tensorflow as tf

import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train.astype('float32') / 255.0

x_test = x_test.astype('float32') / 255.0

def create_model(use_bn=False):

    model = tf.keras.Sequential()

    model.add(tf.keras.layers.Input(shape=(32, 32, 3)))

    model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu'))

    if use_bn:

        model.add(tf.keras.layers.BatchNormalization())
```

```
    model.add(tf.keras.layers.MaxPooling2D((2, 2)))
    model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu'))
    if use_bn:
        model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.MaxPooling2D((2, 2)))
    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(128, activation='relu'))
    if use_bn:
        model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.Dense(10, activation='softmax'))
    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
model_without_bn = create_model(False)
model_with_bn = create_model(True)
print("Training Without Batch Normalization")
history_without = model_without_bn.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
print("\nTraining With Batch Normalization")
history_with = model_with_bn.fit(
    x_train, y_train,
```

```
    epochs=5,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
loss_without, acc_without = model_without_bn.evaluate(
    x_test, y_test, verbose=0
)
loss_with, acc_with = model_with_bn.evaluate(
    x_test, y_test, verbose=0
)print("\nMODEL COMPARISON")
print("Without Batch Normalization")
print("Test Loss:", round(loss_without, 4))
print("Test Accuracy:", round(acc_without * 100, 2), "%")
print("\nWith Batch Normalization")
print("Test Loss:", round(loss_with, 4))
print("Test Accuracy:", round(acc_with * 100, 2), "%")
plt.figure(figsize=(8, 5))
plt.plot(history_without.history['loss'],
         label='Without Batch Normalization')
plt.plot(history_with.history['loss'],
         label='With Batch Normalization')
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss Comparison")
plt.legend()
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(history_without.history['val_accuracy'],
```

```
        label='Without Batch Normalization')
plt.plot(history_with.history['val_accuracy'],
        label='With Batch Normalization')
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.show()
```

INPUT:

CIFAR-10 Dataset:

- 50,000 training images.
- 10,000 testing images.
- Image size: 32 × 32 pixels.
- RGB color images.
- 10 object categories.

Two CNN Models:

1. Without Batch Normalization.
2. With Batch Normalization.

Training settings:

- Epochs: 5
- Batch size: 64
- Optimizer: Adam
- Loss function: Sparse Categorical Crossentropy.

OUTPUT:

```
plt.figure(figsize=(8, 5))

plt.plot(history_without.history['val_accuracy'],
         label='Without Batch Normalization')

plt.plot(history_with.history['val_accuracy'],
         label='With Batch Normalization')

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Validation Accuracy Comparison")
plt.legend()
plt.show()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
 23552000/170498071 ━━━━━━━━━━━━━━━━━━━━ 19:47 8us/step
```

RESULT:

Successfully implemented Batch Normalization in CNN and compared its training loss, validation accuracy and convergence behaviour with a model without Batch Normalization.

Experiment 22: Data Augmentation for Improving Image Classification
Dataset: Kaggle Dogs vs. Cats Dataset

Python Programming Question: Write a Python program using TensorFlow/Keras to apply data augmentation techniques such as rotation, horizontal flipping, zooming, and shifting. Train CNN models with and without augmentation and compare classification performance.

Concept: Data Augmentation | CO2

AIM:

To apply data augmentation techniques such as rotation, horizontal flipping, zooming and shifting using TensorFlow/Keras and compare CNN performance with and without augmentation.

PROCEDURE:

- Import TensorFlow, Keras, NumPy and Matplotlib.
- Load the Kaggle Dogs vs. Cats dataset.
- Resize images to 150 × 150 pixels and normalize them.
- Create a CNN model without data augmentation.
- Create another CNN model with data augmentation.
- Apply rotation, horizontal flipping, zooming and shifting.
- Train both models using the same training data.
- Compare testing accuracy and loss.
- Display the performance comparison graph.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

import os

import glob

from sklearn.model_selection import train_test_split

DATASET_PATH = "/content/dogs-vs-cats/train"

cat_files = glob.glob(os.path.join(DATASET_PATH, "cat.*.jpg"))

dog_files = glob.glob(os.path.join(DATASET_PATH, "dog.*.jpg"))

files = cat_files + dog_files
```

```
labels = [0] * len(cat_files) + [1] * len(dog_files)
train_files, test_files, train_labels, test_labels = train_test_split(
    files, labels,
    test_size=0.2,
    random_state=42,
    stratify=labels
)
def preprocess_image(path, label):
    image = tf.io.read_file(path)
    image = tf.io.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, (150, 150))
    image = image / 255.0
    return image, label
train_data = tf.data.Dataset.from_tensor_slices(
    (train_files, train_labels)
)
test_data = tf.data.Dataset.from_tensor_slices(
    (test_files, test_labels)
)
train_data = train_data.map(
    preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE
)
test_data = test_data.map(
    preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE
)
train_data = train_data.shuffle(1000).batch(32).prefetch(
    tf.data.AUTOTUNE
```

```
)
test_data = test_data.batch(32).prefetch(
    tf.data.AUTOTUNE
)
augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomZoom(0.2),
    tf.keras.layers.RandomTranslation(0.1, 0.1)
])

def create_model(use_augmentation=False):
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Input(shape=(150, 150, 3)))
    if use_augmentation:
        model.add(augmentation)
    model.add(tf.keras.layers.Conv2D(32, (3, 3), activation='relu'))
    model.add(tf.keras.layers.MaxPooling2D(2, 2))
    model.add(tf.keras.layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(tf.keras.layers.MaxPooling2D(2, 2))
    model.add(tf.keras.layers.Conv2D(128, (3, 3), activation='relu'))
    model.add(tf.keras.layers.MaxPooling2D(2, 2))
    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(128, activation='relu'))
    model.add(tf.keras.layers.Dropout(0.5))
    model.add(tf.keras.layers.Dense(1, activation='sigmoid'))
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
```

```
        metrics=['accuracy']
    )

    return model
model_without = create_model(False)
print("Training Without Data Augmentation")
history_without = model_without.fit(
    train_data,
    epochs=5,
    validation_data=test_data
)
model_with = create_model(True)
print("\nTraining With Data Augmentation")
history_with = model_with.fit(
    train_data,
    epochs=5,
    validation_data=test_data
)
loss_without, acc_without = model_without.evaluate(
    test_data, verbose=0
)
loss_with, acc_with = model_with.evaluate(
    test_data, verbose=0
)
print("\nPERFORMANCE COMPARISON")
print("Without Augmentation")
print("Accuracy:", round(acc_without * 100, 2), "%")
print("Loss:", round(loss_without, 4))
print("\nWith Augmentation")
```

```
print("Accuracy:", round(acc_with * 100, 2), "%")
print("Loss:", round(loss_with, 4))
plt.figure(figsize=(8, 5))
plt.plot(history_without.history['val_accuracy'],
        label='Without Augmentation')
plt.plot(history_with.history['val_accuracy'],
        label='With Augmentation')
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Accuracy Comparison")
plt.legend()
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(history_without.history['val_loss'],
        label='Without Augmentation')
plt.plot(history_with.history['val_loss'],
        label='With Augmentation')
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Loss Comparison")
plt.legend()
plt.show()
```

INPUT:

Kaggle Dogs vs. Cats Dataset:

- Contains cat and dog images.
- Two classes: Cat (0) and Dog (1).
- Image size: 150 × 150 pixels.
- RGB color images.

- 80% training images.
- 20% testing images.

Data Augmentation Techniques:

1. Rotation – rotates images.
2. Horizontal Flipping – flips images horizontally.
3. Zooming – enlarges or reduces image size.
4. Shifting – moves images horizontally and vertically.

OUTPUT:

```
plt.plot(history_with.history['val_accuracy'],
         label='With Augmentation')

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Accuracy Comparison")
plt.legend()
plt.show()

# Loss comparison graph
plt.figure(figsize=(8, 5))

plt.plot(history_without.history['val_loss'],
         label='Without Augmentation')

plt.plot(history_with.history['val_loss'],
         label='With Augmentation')

plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.title("Loss Comparison")
plt.legend()
plt.show()
```

PERFORMANCE COMPARISON

Without Augmentation

Accuracy: Calculated value %

Loss: Calculated value

With Augmentation

Accuracy: Calculated value %

Loss: Calculated value

RESULT:

Successfully applied data augmentation techniques and compared CNN classification performance with and without augmentation.

Experiment 23: Transfer Learning Using Pretrained ResNet or EfficientNet
Dataset: Intel Image Classification Dataset (Kaggle)

Python Programming Question: Write a Python program using TensorFlow/Keras to implement transfer learning using pretrained ResNet50 or EfficientNetB0. Replace the final classification layer, freeze pretrained layers, train the modified model using the new image dataset, and evaluate its classification performance.

Concept: CNN Architectures | CO2

AIM:

To implement transfer learning using a pretrained ResNet50 model, replace its final classification layer, freeze the pretrained layers, and classify images in the Intel Image Classification Dataset.

PROCEDURE:

- Import TensorFlow, Keras, NumPy, and Matplotlib.
- Load the Intel Image Classification Dataset.
- Resize images to 224 × 224 pixels and preprocess them.
- Load the pretrained ResNet50 model with ImageNet weights.
- Freeze all pretrained layers.
- Add Global Average Pooling and a new classification layer.
- Train the model using the training dataset.
- Evaluate the model using the test dataset and display accuracy and loss.

PROGRAM CODE:

```
import tensorflow as tf

import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.applications import ResNet50

from tensorflow.keras.applications.resnet50 import preprocess_input

from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense, Dropout

from tensorflow.keras.models import Model

train_dir = "/content/seg_train/seg_train"

test_dir = "/content/seg_test/seg_test"

IMG_SIZE = (224, 224)
```

```
BATCH_SIZE = 32
EPOCHS = 5
train_data = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
val_data = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="validation",
    seed=42,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)

class_names = train_data.class_names
test_data = tf.keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_names=class_names,
    shuffle=False
)
train_data = train_data.map(
    lambda x, y: (preprocess_input(x), y)
```

```
).prefetch(tf.data.AUTOTUNE)
val_data = val_data.map(
    lambda x, y: (preprocess_input(x), y)
).prefetch(tf.data.AUTOTUNE)
test_data = test_data.map(
    lambda x, y: (preprocess_input(x), y)
).prefetch(tf.data.AUTOTUNE)
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
base_model.trainable = False
inputs = Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dropout(0.3)(x)
outputs = Dense(len(class_names), activation="softmax")(x)
model = Model(inputs, outputs)
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=EPOCHS
```

```
)
test_loss, test_accuracy = model.evaluate(test_data)
print("\nClass Names:", class_names)
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
print("Test Loss:", round(test_loss, 4))
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.show()
```

INPUT:

- Intel Image Classification Dataset from Kaggle.
- Training images from seg_train/seg_train.
- Testing images from seg_test/seg_test.
- Image size: 224 × 224 pixels.
- Batch size: 32.
- Number of epochs: 5.
- Pretrained model: ResNet50 with ImageNet weights.

OUTPUT:

```
Class Names: ['buildings', 'forest', 'glacier',
              'mountain', 'sea', 'street']

Test Accuracy: XX.XX %
Test Loss: X.XXXX
```

RESULT:

Transfer learning was successfully implemented using pretrained ResNet50, and the model was evaluated on the Intel Image Classification Dataset.